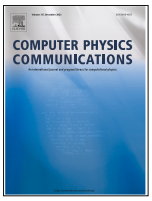
Computer Physics Communications 323 (2026) 110094



Contents lists available at ScienceDirect

Computer Physics Communications

journal homepage: www.elsevier.com/locate/cpc



Computer Programs in Physics



Multi-GPU acceleration of PALABOS fluid solver using C++ standard parallelism

Jonas Latt [a,*], Christophe Coreixas [b,a]

[a] Department of Computer Science, University of Geneva, 1227, Carouge, Switzerland
[b] Institute for Advanced Study, Beijing Normal - Hong Kong Baptist University, 519088, Zhuhai, China

ARTICLE INFO

Editor: Dr. J Ballantyne

Keywords:
Computational fluid dynamics
High performance computing
Palabos
Lattice Boltzmann method
Open-source software
C++ standard parallelism

ABSTRACT

This article presents the principles, software architecture, and performance analysis of the GPU port of the lattice Boltzmann software library Palabos [J. Latt et al., "Palabos: Parallel lattice Boltzmann solver", Comput. Math. Appl. 81, 334-350, (2021)]. A hybrid CPU-GPU execution model is adopted, in which numerical components are selectively assigned to either the CPU or the GPU, depending on considerations of performance or convenience. This design enables a progressive porting strategy, allowing most features of the original CPU-based codebase to be gradually and seamlessly adapted to GPU execution. The new architecture builds upon two complementary paradigms: a classical object-oriented structure for CPU execution, and a data-oriented counterpart for GPUs, which reproduces the modularity of the original code while eliminating object-oriented overhead detrimental to GPU performance. Central to this approach is the use of modern C++, including standard parallel algorithms and template metaprogramming techniques, which permit the generation of hardware-agnostic computational kernels. This facilitates the development of user-defined, GPU-accelerated components such as collision operators or boundary conditions, while preserving compatibility with the existing codebase and avoiding the need for external libraries or non-standard language extensions. The correctness and performance of the GPU-enabled Palabos are demonstrated through a series of three-dimensional multiphysics benchmarks, including the laminar-turbulent transition in a Taylor-Green vortex, lid-driven cavity flow, and pore-scale flow in Berea sandstone. Despite the high-level abstraction of the implementation, the single-GPU performance is similar to CUDA-native solvers, and multi-GPU tests exhibit good weak and strong scaling across all test cases. Beyond the specific context of Palabos, the porting methodology illustrated here provides a generalizable framework for adapting large, complex C++ simulation codes to GPU architectures, while maintaining extensibility, maintainability, and high computational performance.

PROGRAM SUMMARY

Program Title: Palabos: Parallel Lattice Boltzmann Solver
CPC Library link to program files: https://doi.org/10.17632/cx7j83pr69.1
Developer's repository link: https://gitlab.com/unigespc/palabos
Licensing provisions: AGPLv3
Programming language: C++17, using C++ parallel algorithms
Supplementary material: A compressed file containing a snapshot of the described code version
Nature of problem: The lattice Boltzmann method (LBM) is a widely used approach in computational fluid dynamics (CFD) for simulating flows in complex geometries, turbulence, multiphase systems, and thermal transport. The Palabos library was designed to provide the scientific community with a flexible and efficient parallel LBM platform, implementing a broad range of physical and numerical models. Until now, Palabos has been CPU-based, relying on distributed-memory parallelism through MPI. However, with modern simulation platforms and supercomputers increasingly dominated by GPU accelerators, a CPU-only implementation no longer provides competitive performance. This work presents a complete redesign of the Palabos architecture to run efficiently on multi-GPU systems. The new code supports hybrid execution: legacy CPU components remain functional, new GPU-optimized modules can be developed within a compatible programming model, and both can interact

* Corresponding author.
E-mail address: jonas.latt@unige.ch (J. Latt).

https://doi.org/10.1016/j.cpc.2026.110094
Received 21 August 2025; Received in revised form 8 January 2026; Accepted 20 February 2026
Available online 25 February 2026
0010-4655/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

seamlessly. This version supersedes the original CPU-only implementation and will serve as the basis for all future development of Palabos.

Solution method: The GPU version of Palabos introduces a new data container, the `AcceleratedLattice`, designed specifically for accelerator execution. It preserves the user interface of the CPU-based `MultiBlockLattice`, but replaces its array-of-structures layout and virtual-function polymorphism with a structure-of-arrays layout, thread-safe collision-streaming schemes, and integer tags for model selection. These changes enable efficient memory access, fine-grained parallelism, and compatibility with GPU execution models, while maintaining backward compatibility and supporting hybrid CPU/GPU workflows.

Parallelism is expressed through C++17 standard parallel algorithms, compiled with the NVIDIA HPC SDK. Fundamental parallel algorithms such as `for_each`, `transform_reduce`, and `exclusive_scan` are used to implement the essential building blocks of the lattice Boltzmann method, including linear processing, reductions, and data packing for MPI communication. This data-oriented design yields a portable and maintainable GPU implementation, preserves modularity, and ensures that existing Palabos applications can be ported to GPU with minimal modifications.

Additional comments including restrictions and unusual features: The GPU version currently relies on the platform-independent framework of C++ parallel algorithms but has, as of now, only been tested with NVIDIA GPUs (compiled with nvc++) supports NVIDIA architectures with CUDA-compatible compilers (tested with nvc++). The current version of the code does not compile with the 25.X series of the nvc++ compiler.

1. Introduction

Porting computationally intensive scientific software to accelerated hardware can lead to substantial performance improvements. Typical devices used to accelerate scientific applications include field-programmable gate arrays (FPGAs), domain-specific accelerators such as tensor-processing units (TPUs), and Graphics Processing Units (GPUs). The latter have proved particularly successful due to their affordability and relatively broad range of use. Compared to CPUs, GPUs typically offer a substantially higher peak rate of floating-point computations, and a higher peak memory bandwidth, at an equivalent financial cost or energy consumption. Conversely, GPUs can be more challenging to use due to a more restrictive programming model.

General-purpose GPU (GPGPU) programming languages provide a low-level programming interface that is closely tied to the GPU architecture. For instance, CUDA [1] is designed for developing applications that run efficiently on NVIDIA GPUs only, while HIP [2] is more commonly used for AMD GPUs. Alternatively, higher-level approaches have emerged, offering hardware-agnostic solutions that allow, e.g., the same code to run on CPUs and GPUs provided by different vendors. Examples include directive-based coding strategies (OpenACC [3] and OpenMP [4]) and multi-architecture libraries/frameworks (OpenCL [5], SYCL [6], Thrust [7], Kokkos [8]). These approaches enable parts of the code to be offloaded to the GPU with minimal modifications to the original CPU code. A more recent strategy consists in using parallel features available in ISO languages such as C++ and Fortran. This strategy simplifies development and enhances portability by providing a uniform way to write code for both standard CPUs and accelerated hardware. In that context, switching to different target hardware only requires a change of compiler or compilation flag [9,10].

The available C++ parallel algorithms efficiently implement elementary operations, such as search or sorting tasks. Furthermore, generic data-processing algorithms like `for_each` and `transform_reduce` are available, which are customized through user-defined functions and offer a functionality equivalent to a parallelized `for` loop with reductions. These algorithms allow solving a broad range of problems, including stencil operations on a structured grid. We demonstrated this point in [11], which uses C++ parallel algorithms to execute a Computational Fluid Dynamics (CFD) simulation on GPU. The provided open-source demonstration code, based on the lattice Boltzmann method (LBM) and using a homogeneous grid, runs at high performance on GPU. Since the original code implementation, the performance gap of 20% has been narrowed to only a few percent compared to a reference code written in the CUDA programming language. Interestingly, this code demonstrated the feasibility of implementing state-of-the-art scientific software with limited GPU-specific knowledge, and with a portable approach that is exempt from any GPU vendor lock-in and even applies to multiple types of accelerators. Since then, ISO-standard languages have been increasingly adopted to accelerate a wide range of simulation tools on GPUs from various vendors (NVIDIA, AMD, and Intel). These efforts include the acceleration of high-order finite-difference methods [12–14], semi-Lagrangian schemes [15], discrete element methods [16–18]), mini-applications [19–21], and even LB codes running on non-uniform grids [22].

The present article builds on this procedure and proposes a methodology to port an existing CPU-based C++ software library to GPU using parallel algorithms. Particular attention is devoted to the MPI communication layer of the original CPU base, which is reused to allow multi-GPU execution. The resulting code is tested using examples of the LBM software library Palabos [23], many of which were adapted for an execution on a multi-GPU environment prior to drafting this article [24,25]. Despite the wide scope of the Palabos library [26–42], the development effort of the GPU port was accomplished within a few months as a side project [43] and without the assignation of a dedicated developer. This illustrates the light burden of the proposed method, and the adequacy of the method for developers with limited GPU programming knowledge. The characteristics of the GPU port achieved with our method are summarized as follows:

- The existing API of the CPU is maintained to allow previously developed applications to run on both CPU and GPU backends.
- Hybrid CPU/GPU execution is allowed to guarantee access to software components that have not yet been ported or cannot be ported to GPU. Multi-node MPI parallelism is implemented for both CPU and GPU components in a compatible manner to allow fast data exchange.
- Code duplication is minimal, as a common code base is maintained for both CPU and GPU executions.

The rest of the paper is organized as follows. First, Section 2 describes the GPU porting strategy and its application to Palabos fluid solver. Next, Section 3 validates the GPU version of the fluid solver through several benchmarks relevant to its users. Section 4 evaluates its performance on a single A100-SXM4 GPU (40GB) and a full DGX-A100-SXM4 (4×40GB). Finally, Section 5 presents the conclusions regarding the multi-GPU porting of Palabos fluid solver.

All numerical tests in this article can be reproduced using the pre-release version of the GPU-ported Palabos version on the Palabos Web page https://palabos.unige.ch. The test cases are found in the `examples/gpuExamples` directory.

2. Description of the approach

2.1. Overview

In the original version of Palabos, the so-called `MultiBlock` serves as a fundamental container for all parallelized, large data sets. One of its instances, the `MultiBlockLattice`, holds the LBM populations, which are regrouped using a blockwise domain decomposition and assigned to multiple CPU cores and/or multi-processor nodes using MPI primitives. The `MultiBlockLattice` offers a flexible user interface, because (1) the number of populations and lattice connectivity can be adjusted using template metaprogramming, via *lattice descriptors*, (2) the LBM collision model can be adjusted locally using an object-oriented runtime mechanism, through *dynamics objects*, and (3) non-local algorithm ingredients, like the execution of finite-difference stencils, are expressed using the formalism of Palabos *data processors* (additional information is provided in the main Palabos reference [23]). Because of the data organization and execution-model issues described in Section 2.3, accessing the data and associated functionality of the `MultiBlockLattice` directly from within a GPU leads to insurmountable performance issues and even, for some aspects, to technical obstacles that could not be solved with current compiler technology and GPU architecture. Instead, Palabos was enhanced with a new data container, the `AcceleratedLattice`.

To maintain backward compatibility, the `AcceleratedLattice` provides the same user interface as the `MultiBlockLattice`, and in particular, its lattice connectivity can be adjusted using the same lattice descriptors. However, for reasons explained in Section 2.3, it relies on its own implementation of local collision models (formerly *dynamics objects*) and non-local stencils (*data processors*). A large number of such *dynamics objects* and *data processors* have been ported to their GPU counterparts to accompany this GPU release of Palabos, and a formalism has been introduced to support further migration of existing Palabos extensions in a largely automated manner. Furthermore, bridging mechanisms were provided to exchange data between the `MultiBlockLattice` and `AcceleratedLattice` in run time, thus allowing for a hybrid CPU/GPU execution model.

Fig. 1 illustrates the different parallelization strategies applied in the two Palabos data containers, both of which rely on domain decomposition into regular blocks (left part). On the CPU (upper right part), each CPU core processes one or more blocks sequentially, allowing non-local data accesses to be efficiently resolved by a matrix indexing scheme. The granularity of the blocks is determined by the efficiency trade-off between allocating unused cells in large blocks and the need for excessive data transfer in small blocks (see [23] for more details). This approach is however inefficient on GPUs, which typically operate with a larger number of execution threads and thus rely on a finer-grained form of parallelism. Therefore, the operations executed within a block are further parallelized within the GPU (lower right part) using the C++ formalism of parallel algorithms described in Section 2.2. Optimal performance on the GPU is typically achieved by relying on decomposition into larger blocks than on the CPU, so that only a few blocks, or even a single one, are assigned to each GPU. However, despite this different execution mode, both data containers rely on the same MPI-based interblock communication layer. It should be mentioned that the formalism of C++ parallel algorithms can be applied to different types of homogeneous and heterogeneous parallel platforms, although this article focuses on GPUs. In particular, the `AcceleratedLattice` can be used to implement hybrid MPI – shared memory parallelization by applying parallel algorithms to a multi-core CPU. In that case, both a dedicated compiler (e.g. nvc++) or a general-purpose compiler with Intel's Threading Building Blocks library [44] can be used to run an `AcceleratedLattice` on multi-core CPUs.

We finally comment on the learning curve associated with the Palabos GPU port. For applications that rely exclusively on functionality already available in the GPU-enabled code base, most notably standard lattice Boltzmann models and commonly used data processor, the transition is essentially immediate, as it suffices to replace the `MultiBlockLattice` by an `AcceleratedLattice`. For users who have developed custom extensions, such as application-specific dynamics objects or data processors, an explicit port to the new GPU-oriented formalism is required, following the patterns provided by the existing examples. While this entails a non-negligible initial effort, the learning curve is significantly softened by the hybrid design of Palabos, which allows functionalities to be ported incrementally and validated step by step, rather than requiring a full rewrite of the application.

In summary, Palabos-based end-user applications are easily ported to GPU by replacing all reference to a `MultiBlockLattice` to an `AcceleratedLattice`, and by using the ported version of all data processors that are manually invoked, such as finite-difference stencil operators. Palabos also supports a hybrid execution model in which both a `MultiBlockLattice` and an `AcceleratedLattice` data container are instantiated to mirror the simulation results on CPU and GPU, respectively. The data can be copied between either container on a per-need basis to proceed with the computations on the hardware platform on which better suited.

2.2. Programming model

The GPU port of Palabos is realized using C++ standard language parallelism. Available since the C++17 language standard, this hardware agnostic formalism allows compilers to parallelize algorithms of the C++ Standard Template Library (STL) using a shared-memory paradigm chosen by the compiler. Current compiler implementations include GPU and multi-core CPU parallelism. For LBM applications, the formalism showed performance close to that expected from other high-level formalisms, with only a small trade-off in efficiency compared to lower-level approaches such as CUDA [11,22]. The choice of this approach is driven by portability, as it relies solely on intrinsic, ISO-certified language features, and convenience. Thanks to the language integration of this parallel formalism, adapting a code for GPU execution can often be achieved with minimal or no syntactic changes. Nonetheless, a solid understanding of the GPU execution model remains crucial, as discussed in Section 2.3.

The fundamental workhorse for a standard-language parallelization of an LBM-like code are the STL algorithms `for_each` and `transform_reduce`, which essentially express a functional-style *for* loop, with (`transform_reduce`) or without (`for_each`) reductions. The behavior of the `for_each` algorithm can be adjusted by a user-defined operation, equivalent to the body of a *for* loop. This operation is distributed across multiple execution threads when the parallel version of the algorithm is chosen, similar to the effect of parallelizing a *for* loop with the OpenMP formalism. However, the strength of this approach lies in the availability of numerous parallelized algorithms, including algorithms that rely on data dependency between threads. More precisely, the GPU port of Palabos relies on the following algorithms.

Linear Processing. The `for_each` algorithm is used to process data linearly and in-place. Cases where the output container is distinct from the input, the `transform` algorithm is commonly used instead, as a matter of syntactic preference.

Reductions. The `reduce` and `transform_reduce` algorithms are used for all types of reduction operations. Note that due to the severe performance penalty of repeated memory traversals, a `transform_reduce` call is commonly used in Palabos to express both a linear data processing and a reduction operation in a single call.

Prefix Sums. The `exclusive_scan` and `transform_exclusive_scan` algorithms are applied to all types of re-indexing operations, i.e., whenever data is mapped into a larger or smaller container at different index locations. An example includes data packing and unpacking operations

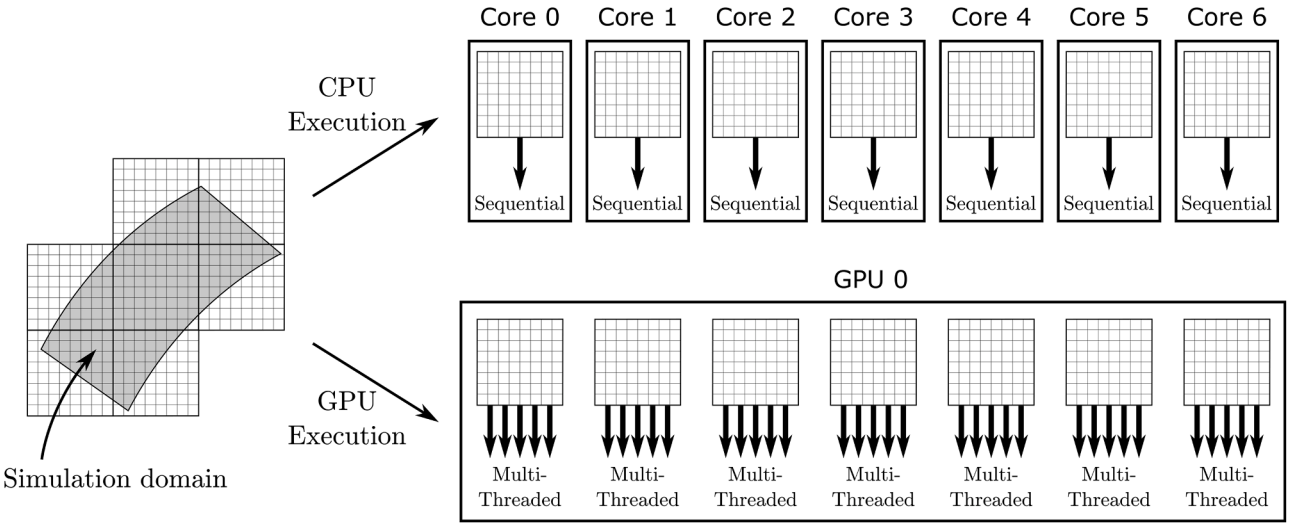


Fig. 1. The grid covering the simulation domain is partitioned into regular blocks. On the CPU, each block is processed sequentially by its assigned CPU core. On the GPU, each block is processed in parallel through the shared-memory formalism of parallel algorithms.

for MPI communication, which must be carried out within GPU memory to guarantee good parallel performance on multi-GPU systems that support direct GPU-to-GPU communication.

2.3. Code design for GPU execution

To port the original Palabos to GPU, it can seem sufficient to apply STL algorithms to parallelize all for loops appearing in its code base. However, this approach falls short due to limitations in data layout, memory access patterns, and execution models, which can result in suboptimal performance or even prevent the code from running on GPU altogether. To properly address the needs of a GPU execution, we introduce a new data container, the `AcceleratedLattice`, designed specifically for GPU execution, alongside the existing `MultiBlockLattice`. This new structure promotes uniformity in both data access and execution flow. While the original Palabos framework follows an object-oriented paradigm, we use the terminology of *data-oriented design* to refer to the GPU-optimized version. This shift places primary emphasis on how data is organized and accessed, with higher-level abstractions introduced only afterward to ensure maintainability and adherence to sound software engineering principles. The fundamental ideas of this design are described in this section.

2.3.1. Structure-of-array memory layout

One of the key differences between the `AcceleratedLattice` and `MultiBlockLattice` containers lies in the choice of data structure. The `MultiBlockLattice` chooses an array-of-structure (AoS) layout, which is well suited to the object-oriented approach of the original Palabos code, because it places all data belonging to the same grid node at successive memory addresses. Nonetheless, it is well known [45] that a structure-of-array (soa) layout is better suited for GPU-based LBM implementations. Our previous work [11] notably shows that within the formalism of C++, a four-fold (or even higher) speedup can be achieved by switching from AoS to soa.

Fig. 2 illustrates the two implementation strategies in a fictional example with four data elements per grid node, using a different color for each type of data element (or population index). In the LBM, this number is actually larger, with typically 19 or 27 populations. To understand the performance difference between the two approaches, it is necessary to focus on the streaming step, which is less naturally inclined to achieve optimal performance due to the complex memory access pattern. As shown in Fig. 2, an SoA implementation strategy groups all variables that communicate during the streaming step (e.g., all red cells or all f_1 populations) into a contiguous block. Thanks to this, the regularity of the memory access scheme is greatly enhanced, thus explaining the impressive GPU speedups obtained with the SoA data layout for LBMs. On CPUs, however, the situation is less clear-cut, and the preferred data layout depends on the type of CPU and the number of cores [11].

It is also worth noting that the `MultiBlockLattice` executes the collision-streaming cycles in a thread-unsafe manner within a single block using a fused in-place swap algorithm, which is unproblematic in the single-threaded execution per-block operations of the original Palabos library. For the multi-threaded execution model of the `AcceleratedLattice`, it is necessary to switch to a thread-safe approach, several of which (two-population, AA-pattern, swap with separate collision and streaming, esoteric twists) are presented in Refs. [11, 46]. The current GPU port of Palabos uses the two-population scheme, which doubles the memory requirements for the populations but guarantees performances compatible with other approaches. Future work will consider switching to a non-fused swap approach, such as the AA pattern, to halve the memory footprint without significantly impacting performance, albeit at the cost of a more complex implementation.

2.3.2. Collision-model polymorphism without virtual function calls

The `MultiBlockLattice` in Palabos adopts a natural object-oriented model in which every grid node is represented by an object that regroups all data attributed to the node, including the populations. The collision step is executed polymorphically through virtual function-call resolution, allowing the collision to be adjusted locally during run time. Furthermore, Palabos users can design user-specific collision terms by designing their own class of type `Dynamics`, allowing to freely adjust the behavior of their simulation on a per-cell basis. This choice caters to the typical needs of the LBM community by allowing variable physical and numerical properties in space and time, and defines the versatility of the Palabos library. However, it presents several challenges for the GPU port. The first issue arises from the incompatibility of data layouts: the AoS layout naturally adopted in an object-oriented approach conflicts with the SoA layout required for efficient GPU implementation. Secondly, virtual functional-call resolution did not appear as an acceptable mechanism for the GPU port, as it was not supported by the compiler of the NVIDIA HPC SDK used for the tests and benchmarks of this work. Even if this technique was supported in future compiler versions, it would fundamentally conflict with performance considerations on the rather simple execution model of GPUs.

Switching from an AoS to an SoA data layout for the `AcceleratedLattice` presents a challenge for code reuse. This is

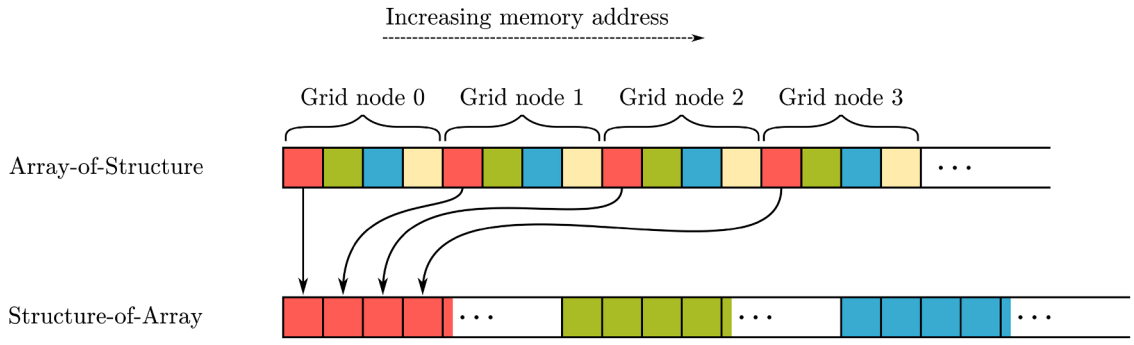


Fig. 2. Array-of-structure versus structure-of-array data layout in an example with four data elements per grid node.

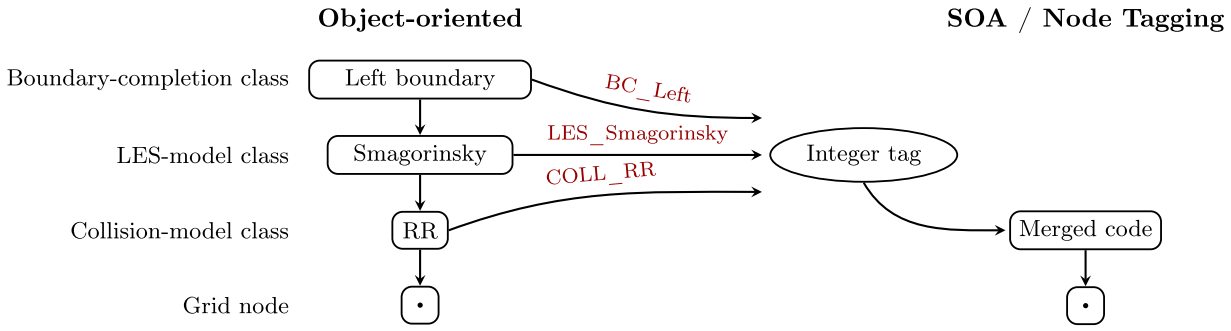


Fig. 3. A succession of polymorphic objects in the object-oriented structure translates to a unique integer identifier for the data-oriented data structure, exploiting Palabos's built-in unique string identification of numerical model classes.

particularly true for the significant portion of the Palabos code handling various collision models and other cell processing steps. This problem was overcome using a classical push-pull implementation of the collision and streaming steps. This principle implies that the data of a node that is scattered in GPU memory following the SoA layout of the `AcceleratedLattice` is pulled into local, adjacent variables whenever a node is processed in the collision-streaming cycle, after which original Palabos code portions are applied to the local node. In such a way, unnecessary code duplication between the original and GPU-ported parts of Palabos is avoided.

As a replacement for the virtual function-call approach in `MultiBlockLattice`, the `AcceleratedLattice` assigns an integer tag to every node, which is used to invoke the appropriate collision model using a `switch`-like language construct. The implemented formalism provides a one-to-one correspondence between the new tag and the original `Dynamics` object, to allow the `MultiBlockLattice` and the `AcceleratedLattice` to mirror each other for pre- and post-processing tasks, or for a CPU/GPU-hybrid program execution. In this context, a run-time type identification (RTTI) mechanism is required, both to assign a unique identifier to each collision model and to allow automatic instantiation of the respective object-oriented and data-oriented structures. Given the limited RTTI support of the C++ language, our method relies on additional library-level RTTI functionality. In Palabos, the mentioned unique integer tags are complemented with string identifiers, which were originally implemented for network transfer and checkpointing of serialized data (see Ref. [23]). Depending on the context, different identification methods may be preferred. Tag-based identification is more efficient and suited for tasks like collision-streaming kernel execution, while string-based identification is more user-friendly and used in cases such as custom kernel generation (see Section 2.3.2).

In Palabos, a specific difficulty stems from the handling of so-called *chained collision models* that are frequently used in a `MultiBlockLattice` and arise when a chain of events is composed through a sequence of `Dynamics` objects that reference each other in a linked-list fashion. A grid node may for example implement a given basic collision model, such as the recursive-regularized (RR) collision [47,48], but also rely on a turbulence model like the Smagorinsky subgrid-scale model [49,50], and further depend on an added completion algorithm for unknown variables on a boundary node. In the AcceleratedLattice, this chain must be identified through a unique tag, which is automatically assigned by the GPU framework of Palabos. The integer tag is deduced from the index of an ordered list of string identifiers, which themselves are concatenated versions of the string identifiers of individual model components, as defined by the object-oriented Palabos version. The underlying mechanism is illustrated on Fig. 3.

A final aspect tackled by the Palabos framework responsible for the correspondence between the MultiBlockLattice and the AcceleratedLattices's polymorphic behavior is the storage of collision-specific model parameters. For example, a node implementing a Smagorinsky subgrid-scale LES closure needs access to the Samgorinsky constant, which, in the original version of Palabos, is stored within the corresponding `Dynamics` object. Consistently with the SoA vision of the new GPU framework, the AcceleratedLattice allocates a separate array for the model parameters of all nodes. Additionally to the integer tag, the nodes now store an index into this array to access additional dynamic content on a per-need basis.

Fig. 4 illustrates the relationship between these two ways to offer Palabos users per-cell customization of the collision model. It is possible for a GPU-based Palabos application to rely solely on the `AcceleratedLattice`, which occupies the right space on the figure. Alternatively, the user may set up the simulation on CPU, using the original `MutliBlockLattice` (left part of the image) and copy all the data, including the specification of the collision models, to the accelerator-friendly container when the initialization is complete. This is especially useful to rapidly port existing Palabos end-user applications to GPU. Then, during program execution, both parallel containers can seemlessly exchange data, including dynamically adapted collision models. Interestingly, the backend which facilitates this exchange represents the major part of the work spent into formulating the Palabos GPU port. While this backend automatizes the work of porting all existing collision mod-

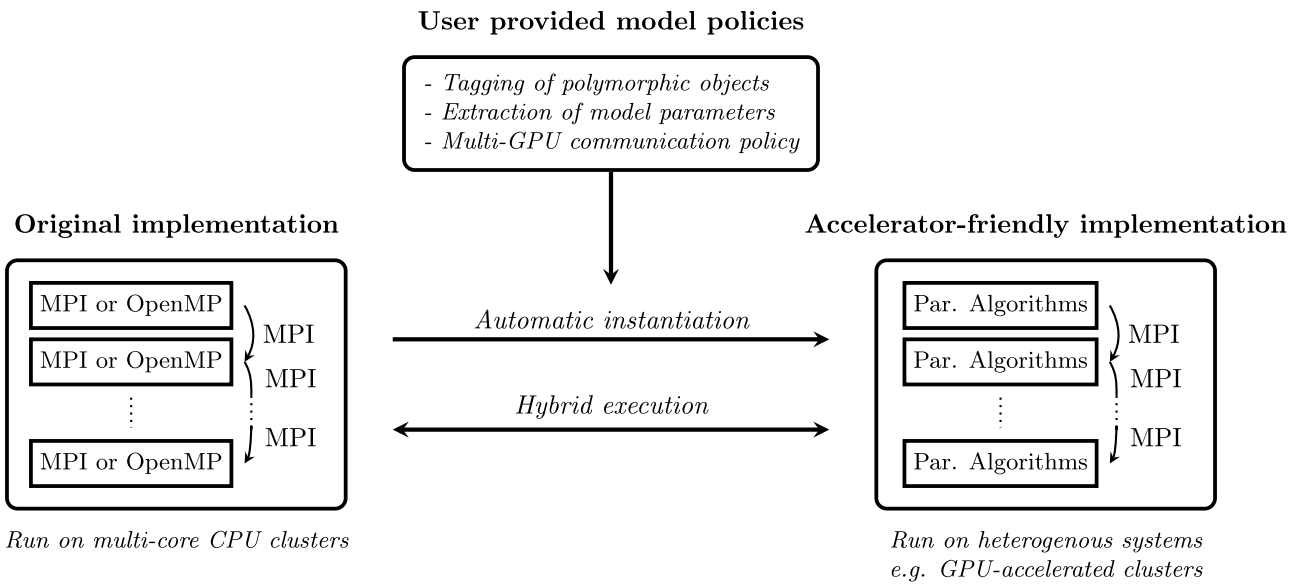


Fig. 4. Illustration of the framework developed to (1) create accelerated data structures (`AcceleratedLattice`) from Palabos's original object-oriented data structures (`MultiBlockLattice`), and (2) use both of them in an hybrid fashion.

els to GPU, an additional manual work is required for every model in the form of user-provided model policies that provide meta-information on the model (central part of Fig. 4), for the extraction of model parameters and serialization of the data in view of MPI communication. This work has been performed for many models available in Palabos but is yet to be done by Palabos users who wish to run their own custom models to GPU.

GPU kernel size reduction

As an undesired side-effect of using an integer-tag approach to handle the polymorphic behavior of nodes in the `AcceleratedLattice`, the GPU kernel executed for the collision-streaming cycle can reach a considerable size, as it must contain the code for all models that have been assigned a tag. The occupation of GPU memory by this kernel impacts the performance of the code considerably, and unnecessarily, as a typical simulation only uses a limited number of models. This leads to a sizeable performance impact in the current version of the code, as approximately 30 collision models have been ported, and a further performance loss is expected from the ongoing implementation efforts.

To circumvent this problem, the collision-streaming function of the `AcceleratedLattide` was adapted to allow hand-picking the models that are compiled into the GPU kernel, with the help of template metaprogramming techniques and the possibility of C++ to offer variadic template parameters. The following code extract illustrates the approaches using both a complete kernel (all collision models) and a custom kernel (hand-picked collision models):

```
// Plain function call, includes all collision models in the kernel.
acceleratedLattice.collideAndStream();

// Custom function call with handpicked
   collision models.
acceleratedLattice.collideAndStream (
    CollisionKernel<T,DESCRIPTOR,
        CollisionModel::NoDynamics,  // For empty nodes
        CollisionModel::TRT,         // For standard bulk nodes
        CollisionModel::BounceBack,  // For no-slip solid nodes
        CollisionModel::Boundary_RegularizedVelocity_0_1__TRT,
                    // Exterior boundaries
        CollisionModel::Boundary_RegularizedVelocity_0_M1__TRT>() );
```

In this example, taken from the Berea sandstone benchmark in the validation section, the two final collision models are composite, and are automatically instantiated by Palabos to implement the inlet and outlet boundary conditions. The example shows that it is not always simple for Palabos users to identify the name of all collision models needed by the kernel. Also, missing models cannot be spotted by the compiler and need to be extracted through meticulous work from linker error messages. As a workaround, Palabos offers a function called `showTemplateArguments(MultiBlockLattice const&)` which identifies all required model names through introspection of a `MultiBlockLattice` and produces a Palabos code with the corresponding custom-kernel call to `collideAndStream` function. In practice, a new Palabos end-user application therefore needs a first run to retrieve the proposer code of the collision-streaming function call. After inclusion of the proposed code, the application can be compiled for the production run.

GPU-GPU communication using pinned memory and MPI

Multi-GPU performance on modern accelerator nodes for memory-bound solvers is dominated by halo exchange efficiency, making direct intra-node GPU-to-GPU communication a prerequisite for parallel scalability. On the NVIDIA DGX A100 platform used in this work, NVLink/NVSwitch provides a high-bandwidth interconnect (up to 600 GB/s aggregate GPU-to-GPU bandwidth [51]), which is sufficient for the relatively small data volumes exchanged during LBM halo updates, though still lower than on-device GPU memory bandwidth. Any fallback to host memory introduces CPU-GPU transfers with bandwidths orders of magnitude smaller, severely degrading performance. To avoid this, halo buffers must be pinned to the GPU memory, allowing MPI to operate directly on device-resident data without intermediate copies.

Concretely, this is achieved using a *CUDA-aware MPI* implementation, which can send and receive GPU buffers directly. Open MPI defines CUDA-aware support as the ability for the MPI library to operate on device memory, typically via UCX as the transport layer [52]. In UCX-based stacks, intra-node GPU-to-GPU transfers use CUDA inter-process communication mechanisms (`cuda_ipc`) to achieve high-performance peer-to-peer transfers across the GPU fabric [53,54].

While Palabos primarily relies on ISO C++ standard parallelism (`stdpar`) for portability, a small non-standard extension is necessary to achieve optimal GPU-GPU communication. Standard C++ containers such as `std::vector` allocate pageable host memory, which forces implicit CPU-GPU memory copies during MPI communication, hence severely limiting parallel speedup even at the scale of a single GPU node. To address this, halo buffers are allocated explicitly as CUDA device memory (`cudaMalloc`), ensuring that MPI receives device

pointers and can leverage its CUDA-aware fast paths. Following best practices [25,52], this workaround is restricted to communication buffers, and it is expected to become unnecessary as C++ and MPI provide improved heterogeneous memory support in the near future.

3. Validation

This section presents validations of the accuracy and performance of the GPU execution of selected benchmarks cases, all of which are available in the latest release of the Palabos software. These benchmark applications differ from classical Palabos end-user applications through the use of the data-oriented `AcceleratedLattice` in place of the object-oriented `MultiBlockLattice`, or where appopriate, the simultaneous use of both data containers to warrant a hybrid execution model. The quality of the GPU port is investigated hereafter through several CFD problems of increasing complexity. Being a good compromise between accuracy, robustness and efficiency, all simulations are performed using the D3Q19 lattice – other velocity discretizations will be included in future releases. Depending on the test requirements, appropriate collision models will be chosen among those already available in Palabos. If not otherwise stated, the convective scaling is adopted so that both the Reynolds and Mach numbers remain unchanged when moving from one mesh resolution to another [55]. Solid and inlet boundary conditions are enforced through Dirichlet conditions based on the standard and modified bounce-back conditions respectively [56,57]. All simulations are performed using single precision, while improving the floating-point accuracy by striping off the constant offset from the populations, as described in the seminal work by Skordos [58].

3.1. Transitional flow: Taylor-Green vortex

The transition from a laminar state to a turbulent regime is very common in an airflow, as its kinematic viscosity is of the order of $10^{-5}[m^2/s]$. Hence, it is crucial for aerodynamics simulations to capture such transitions accurately. The Taylor-Green vortex is one of the simplest tests to investigate this phenomenon since it neither requires complex initial state nor boundary conditions [59] and is therefore frequently used to benchmark the accuracy of CFD solvers [60–63].

This test consists of the interaction of several vortices that are initialized in a periodic box using the following analytical formulas:

$$p = p_\infty + \frac{\rho_\infty u_\infty}{16}\big[\cos(2z)+2\big]\big[\cos(2x)+\cos(2y)\big] \tag{1}$$

$$u_x = \ u_\infty \sin(x)\cos(y)\cos(z) \tag{2}$$

$$u_y = -u_\infty \cos(x)\sin(y)\cos(z) \tag{3}$$

$$u_z = 0 \tag{4}$$

As the simulation starts, the vortices interact and generate smaller vortices through a turbulent energy cascade. This continues until the smallest vortices reach the Kolmogorov scale, at which point they are dissipated through viscous effects.

In this work, $p_\infty = \rho_\infty c_s^2$, where $\rho_\infty = 1$ and $c_s^2 = 1/3$ in lattice units. Furthermore, we will focus on the configuration at Re = 1600 for which DNS results are available [64]. The Mach number is fixed to Ma = 0.2 at all resolutions, which means that the increase of resolution is imposed with convective scaling. To investigate the laminar-to-turbulent transition and the turbulence decay, it is common to monitor the time evolution of the kinetic energy k and the enstrophy ϵ averaged over the entire domain. In our case, these quantities were computed as follows:

$$k = \frac{1}{\Omega}\int_\Omega \frac{u^2}{2}\,\mathrm{d}\Omega, \quad \epsilon = \frac{1}{\Omega}\int_\Omega \frac{\omega^2}{2}\,\mathrm{d}\Omega, \tag{5}$$

where the size of the simulation domain Ω is $[2\pi L]^3$ with L the number of points in each direction. The vorticity $\boldsymbol{\omega} = \boldsymbol{\nabla}\times\boldsymbol{u}$ is calculated using an *eighth-order* centered finite difference approximation not to introduce numerical dissipation during post-processing.

This benchmark is performed using a large number of collision models: the single-relaxation-time BGK, multi-relaxation-time collision applied in either raw (RM), Hermite (HM), central (CM), central Hermite (CHM) or cumulant (K) spaces, and the recursive regularized (RR) approach. While the first model relies on the second-order weighted equilibrium, the others are all based on the fourth-order non-weighted equilibrium, detailed in Eqs. (H15)-(H21) of our previous work [48]. To improve the robustness of LBMs in under-resolved conditions, relaxation times related to the bulk viscosity and higher-order moments are fixed to 1.0 for all collision models except BGK – see Refs. [11,65] for the exact methodology.

We report in Fig. 5 results obtained for increasing mesh resolutions $L \in \{128, 256, 512\}$ and the above-mentioned collision models. As expected, all models lead to the accurate estimation of the kinetic energy and enstrophy evolution over time for $L = 512$. By progressively reducing the mesh resolution from $L = 512$ to 128, the collision step introduces more numerical dissipation. This notably leads to underestimating the enstrophy peak at $t \approx 8t_c$, with the convective time being defined as $t_c = 2\pi N/u_\infty$. Since BGK introduces the least amount of numerical dissipation [63,66,67], it naturally leads to the most accurate results. However, low numerical dissipation implies stability issues for coarse mesh resolutions ($L \leq 128$), hence the unphysical second enstrophy peak observed at $t \approx 12t_c$, as the BGK model reaches its stability limit for $L \leq 128$.

Fig. 5 presents results obtained for the aforementioned collision models, and increasing mesh resolutions $L \in \{128, 256, 512\}$. As expected, all models accurately estimate the kinetic energy and enstrophy evolution over time for $L = 512$. However, as the mesh resolution is progressively reduced from $L = 512$ to 128, the collision step introduces increasing numerical dissipation. This leads to a notable underestimation of the enstrophy peak at $t \approx 8t_c$, where the convective time is defined as $t_c = 2\pi L/u_\infty$. Since BGK introduces the least numerical dissipation [63,66,67], it naturally produces the most accurate results. However, low numerical dissipation can cause stability issues at coarse mesh resolutions. This results in an unphysical second enstrophy peak at $t \approx 12t_c$, as the BGK model reaches its stability limit for $L \leq 128$.

3.2. Wall-bounded flow: Lid-driven cavity

Wall-bounded flows are prevalent in many engineering applications, such as aerodynamics, where the flow over aircraft surfaces needs to be accurately simulated, and in hydraulics, where the behavior of fluid in pipelines and around obstacles is of particular interest.

Hereafter, such flows are used to validate the implementation of no-slip and moving boundary conditions in the GPU version of Palabos. Among them, the lid-driven cavity flow is a perfect candidate as it involves simulating a viscous fluid flow enclosed in a cubic cavity of size L [11,68–73]. In this setup, all walls are fixed except for the top lid, which moves at a constant velocity $u_x^{(z=L)} = c_s\mathrm{Ma}$. The flow inside the cavity exhibits various characteristics depending on the Reynolds number. At lower Reynolds numbers, the flow tends to be steady and laminar. As the Reynolds number increases, the flow becomes unsteady, displaying a complex interplay of boundary layers, vortices, recirculation zones, and sometimes vortex shedding.

We consider three flow configurations of increasing complexity: steady flow at Re = 1000, unsteady laminar flow at Re = 3200, and unsteady turbulent flow at Re = 10000. For Re ≤ 3200, we use a mesh resolution of $L = 256$ cells and the standard BGK collision model with second-order weighted equilibrium. For the turbulent case, we adopt the more robust RR collision model and set the relaxation time related to bulk viscosity to unity to ensure stability with the coarse resolution of $L = 400$ cells. All simulations are conducted at a Mach number of Ma = 0.1.

Velocity profiles along the centerlines of the cavity (plane $y = L/2$) are shown in Fig. 6. For Re = 1000, the flow is steady, and measurements taken after the flow has stabilized (around $78t_c$) match perfectly

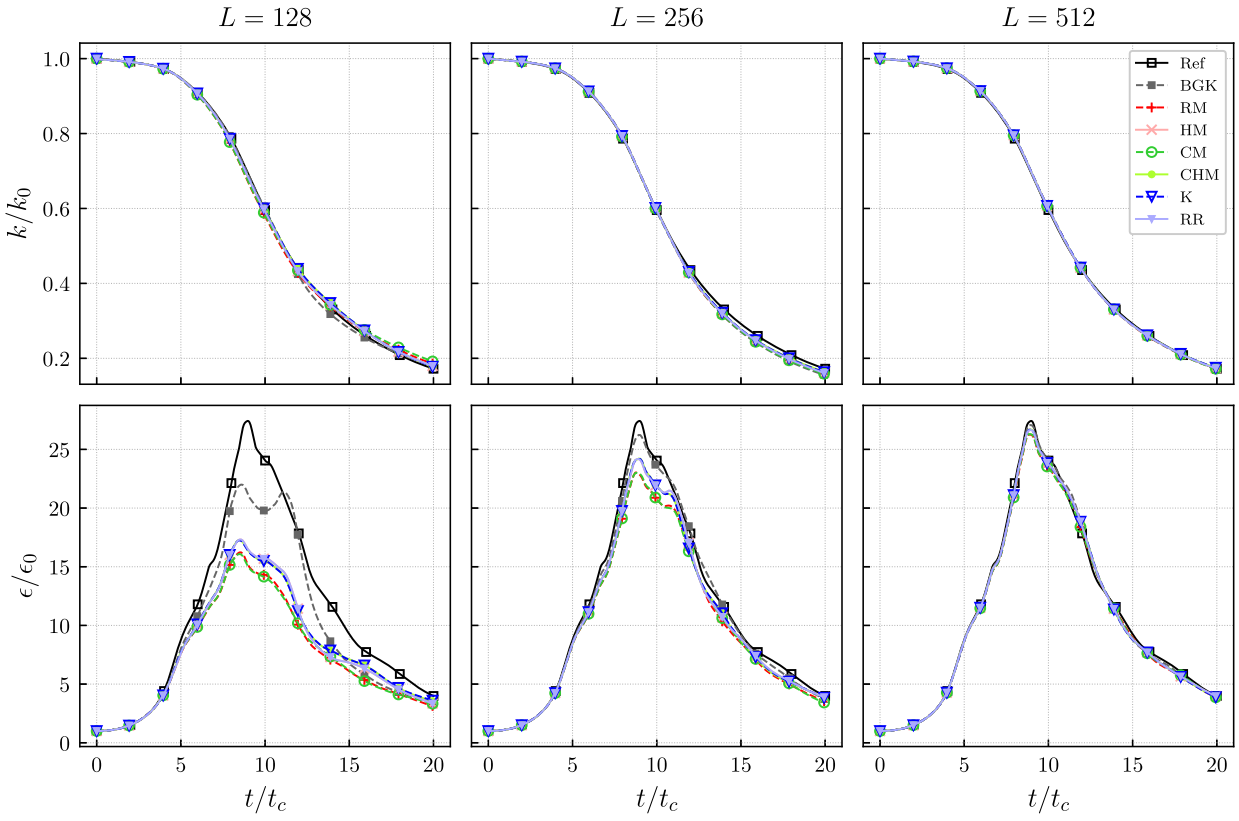


Fig. 5. Taylor-Green vortex simulations for Re = 1600, Ma = 0.2, various mesh resolutions $L \in \{128, 256, 512\}$, and most collision models implemented in Palabos. Time evolution of the normalized kinetic energy k/k_0 (top) and enstrophy ϵ/ϵ_0. The subscript 0 stands for quantities at $t = 0$, and the convective time is defined as $t_c = 2\pi L/u_\infty$.

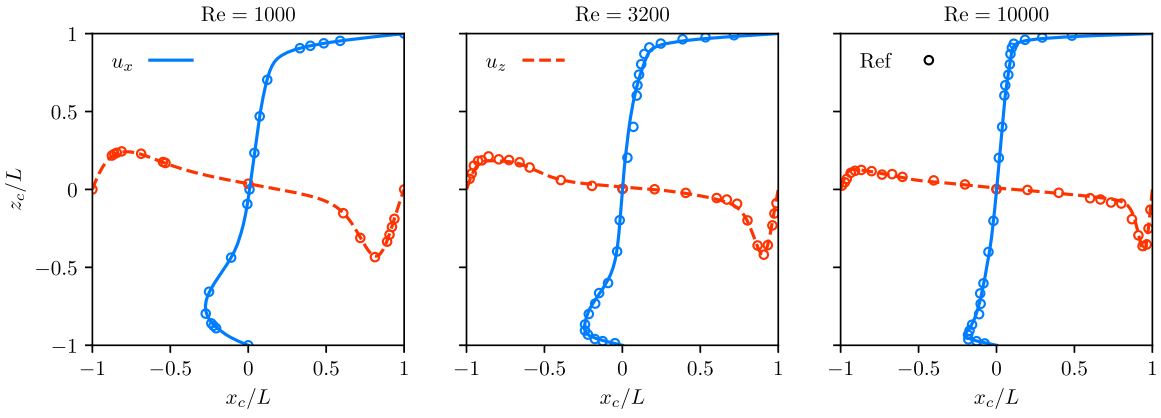


Fig. 6. Lid-driven cavity flow simulations at various Reynolds numbers and a fixed Mach number of 0.1. Results are plotted along the two center lines of the cavity (slice $y = L/2$) using normalized coordinates $\alpha_c/L = 2\alpha/L - 1$ with $\alpha = x$ or z. From left to right panels: steady results at Re = 1000, non-stationary laminar flow at Re = 3200, and turbulent flow at Re = 10000. Reference data (symbols) come from the spectral code [69] and experimental data [68] for $Re = 1000$ and $Re \geq 3200$ respectively.

with simulation data from a spectral code [69]. The flow becomes unsteady when the Reynolds number increases to Re = 3200. Therefore, velocity profiles must be averaged before any comparison with reference data. Averaging is performed every convective time unit ($t_c = L/c_s\text{Ma}$) from $t = 50t_c$ to $t = 250t_c$. The resulting averaged profiles are in excellent agreement with experimental results [68]. The flow becomes more chaotic at the highest Reynolds number (Re = 10000), as the complex interplay between vortices and boundary layers emerges. Hence, velocity profiles are averaged over a longer period, from $t = 150t_c$ to $t = 500t_c$, with the same output frequency. This final comparison also shows a very good match with the experimental data [68]. Consequently, these results confirm the accurate and robust implementation of bounce-back-type boundary conditions in the GPU version of Palabos.

3.3. *Pore-scale fluid flow through porous rocks: Berea sandstone*

Accurate characterization of fluid flow through porous rocks is essential for determining geological properties such as permeability. Numerical simulations support applications including enhanced oil recovery optimization [74–76] and groundwater transport modeling [77–79]. Advances in micro-CT imaging have led to larger and higher-resolution digital samples, demanding significant computational resources for fluid dynamics analysis.

Building on the extensive use of the original CPU-based version of Palabos in this field (see, e.g., Refs. [80–86], among others), the new multi-GPU implementation provides a scalable and efficient solution to address the growing computational demands of high-resolution porous media simulations. To demonstrate its capabilities, we perform pore-scale simulations of single-phase flow through a Berea sandstone sample –a well-characterized porous medium. The sample, referred to as sample B by Dong et al. [87], was acquired via CT scanning at a resolution of $\delta x = 5.345, \mu$m per voxel on a $400 \times 400 \times 400$ grid, yielding a porosity of 19.6% (the full dataset is available at [88]).

This case provides a valuable test for the GPU-ported version of Palabos, as the heterogeneous domain presents unique challenges for GPU computation. It also demonstrates hybrid CPU/GPU workflows, where pre- and post-processing remain CPU-based due to their current complexity. Additionally, the benchmark highlights the application of velocity- and pressure-driven boundary conditions using regularized conditions [89], extending beyond the standard bounce-back approach at the inlet and outlet.

As shown in Fig. 7, the simulation domain spans $(L + 80) \times L \times L$ cells, partitioned into upstream $(40 \times L \times L)$, porous rock $(L \times L \times L)$, and downstream $(40 \times L \times L)$ sections. Inflow and outflow conditions are applied at the upstream and downstream regions, respectively. The mesh resolution matches the digital sandstone dataset, with $L = 400$ cells. The simulation is validated by computing the absolute permeability:

$$k = \frac{\bar{u} \cdot \nu \cdot lx}{\Delta P}, \tag{6}$$

where $\bar{u}$ is the mean velocity along the flow direction, ν the kinematic viscosity, $lx = L, \delta x$ the porous medium length, and ΔP the pressure drop across the domain. Permeability is reported in millidarcies $(1, \text{mD} = 9.869233 \times 10^{-16}, \text{m}^2)$. Two approaches are employed: pressure boundaries at inlet and outlet, and velocity boundaries, both using regularized conditions [89]. Simulations proceed to steady state, after which either the pressure drop (velocity boundaries) or average flow rate (pressure boundaries) is measured. Average flow is computed within the porous rock, including zero velocity for solid nodes, though comparable results are obtained using upstream or downstream measurements. The D3Q19-TRT lattice Boltzmann model with double precision is used for fluid nodes, while solid nodes apply a standard bounce-back condition.

The left panel of Fig. 8 presents the permeability results obtained with both boundary condition approaches at decreasing Reynolds numbers. The relaxation time is fixed at $\tau = 1$, while the lattice velocity u_{lb} is progressively reduced to minimize inertial effects and approach the Stokes flow regime. This also reduces the Mach number, mitigating compressibility and finite time step errors. Both methods yield a converged permeability of 1785.2 mD, with agreement to all five reported digits. Even at the highest Reynolds number simulated, the permeability barely deviates from the converged value by less than a percent.

The simulated permeability of 1785.2 mD can be compared with the 1360 mD estimated by pore-network modeling in the work by Dong et al. [87], representing a difference of 31.2%.

This agreement should be considered satisfactory, given the well-known challenges in estimating the permeability of Berea sandstone, which is highly sensitive to both sample variability and numerical method details. As demonstrated by Kang et al. [90], permeability can vary by several orders of magnitude even for samples with similar porosities. This sensitivity is further confirmed in our results: modifying the domain size L causes only slight changes in porosity but leads to significant variations in the measured permeability (see Table 1).

Another objective of this type of simulation is to reach steady state as fast as possible to speedup the prediction of permeability. Since LBM is explicit in time, it inherently requires time-dependent evolution to achieve stationarity. However, it is also possible to optimize numerical parameters to accelerate convergence. The right panel of Fig. 8 illustrates that through the permeability convergence over time iterations for both pressure-gradient-driven and velocity-driven flows at Re = 0.0024,

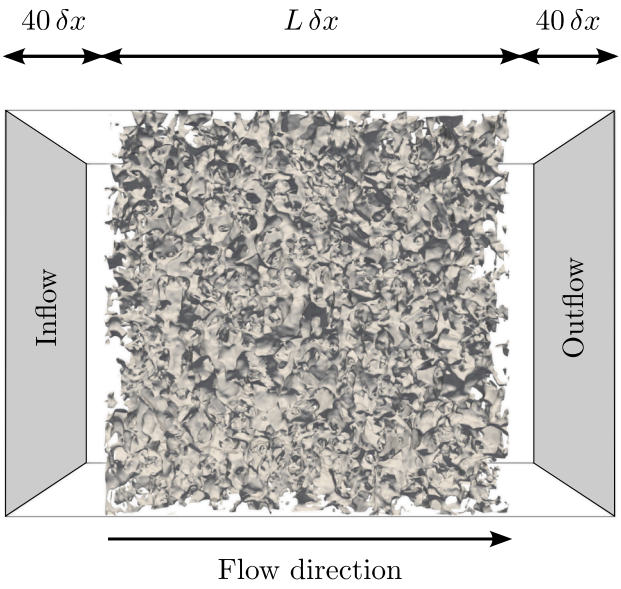


Fig. 7. Fluid flow through Berea sandstone. Illustration of the simulation domain, with the porous rock at its center, and upstream/downstream subdomains. The simulation is performed with the TRT collision model, and a mesh resolution of $L = 400$ using double precision.

Table 1
Berea sandstone properties obtained via a pore-network approach (Dong et al., 2009) and present LBM simulations. The LBM results show that permeability can vary significantly, even for similar porosity values, depending on the domain size L.

L	Porosity (%)	Permeability (mD)	
		LBM	Pore-network
300	20.41	1702	–
350	19.94	1820	–
400	19.64	1785	1360
450	19.79	1443	–
500	19.86	1482	–

plotted on logarithmic axes due to the slow convergence behavior. For velocity-driven simulations, the time step (and corresponding Mach number) is systematically reduced by an order of magnitude, while the relaxation time τ is adjusted to maintain a constant Reynolds number. This tuning is feasible primarily because the TRT collision model decouples wall positioning from τ; without this, time-step convergence of the permeability would not be as straightforward, especially with the BGK collision model.

The results indicate that for $\tau = 1$, the pressure-gradient-driven case reaches five-digit accuracy of the permeability within20ı000 iterations, whereas the velocity-driven case initially requires up to400ı000 iterations. However, further reduction of the lattice velocity and relaxation time enables the velocity-driven case to match the convergence rate of the pressure-gradient approach. In both cases, the permeability of the $400 \times 400 \times 400$ Berea sandstone sample is determined in *under 10 min using a single A100-SXM4 (40GB) GPU*.

4. Performance study

Hereafter, we analyze the performance of the multi-GPU version of Palabos and compare it to the same code running on CPU. The GPU and multi-GPU benchmarks were performed on a NVIDIA DGX A100 workstation equipped with four A100-SXM4 (40 GB) GPUs, interconnected via NVLink 3 in a fully connected topology. Each GPU delivers

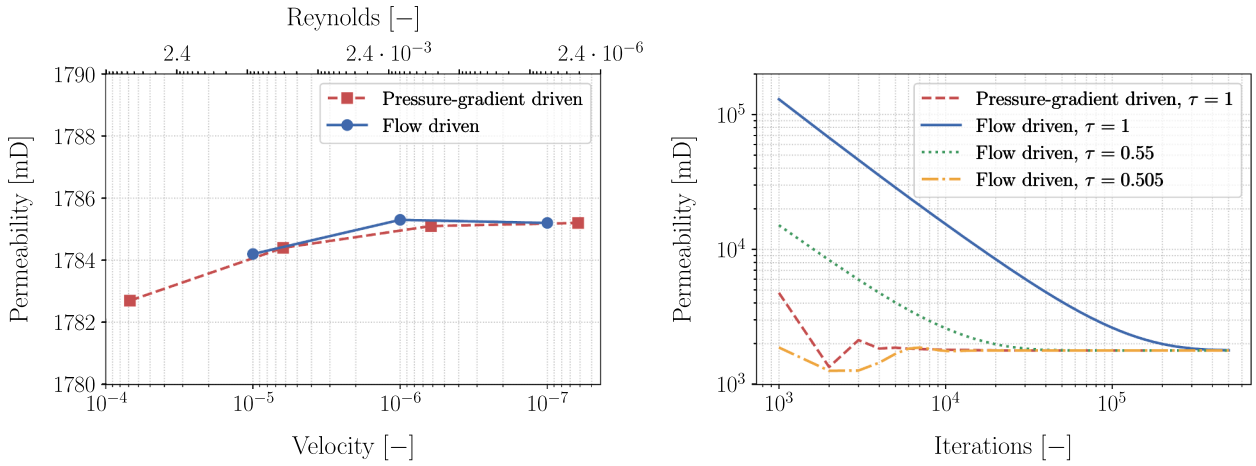


Fig. 8. Permeability computed at fixed relaxation time $\tau = 1$. Left panel: Result with pressure and velocity condition for permeability vs. velocity and Reynolds number. Right panel: Convergence of permeability at a fixed Reynolds $Re = 0.0024$ with different boundary conditions and numerical parameters.

a theoretical memory bandwidth of 1.555 TB/s, while the NVLink 3 interconnect provides up to 600 GB/s of aggregate GPU-to-GPU bandwidth [51]. GPUs were operated at default application clocks with a 400 W power limit and ECC enabled. All GPU benchmarks were compiled with NVIDIA HPC Compiler 25.3 using the `-stdpar` option for GPU offloading, `-O3` and `-DNDEBUG` for production optimization, and `-Msingle -Mfcon` for improved single-precision performance. CPU benchmarks were performed on a dual-socket Intel Xeon Gold 6240R system (48 cores in total) running at 4 GHz, compiled with GCC v13.3 using `-O3` and `-DNDEBUG` optimization flags.

4.1. Performance model and memory usage

Building a performance model is the first step toward understanding how close the implementation of a numerical method is to achieving its peak performance. In the context of GPU acceleration, the available computational power commonly exceeds the one of a CPU by several orders of magnitude. Because of this, the collision step of LBMs is almost instantaneous, and most of the simulation time is spent reading LB data from memory and writing it back after the streaming step. This is why LBMs are commonly referred to as *memory-bound* solvers when running on cluster-class GPUs [11,22,46,91].

For memory-bound solvers, peak performance can be estimated using the bandwidth of GPUs. For the A100-SXM4 (40GB), the maximum bandwidth is $B_{\text{A100}} = 1.555$ GB/s according to the specifications provided by NVIDIA. Dividing B_{A100} by the amount of data accessed during one time iteration n_t, one ends up with the maximum number of cells (or lattices) that can be processed per second. Since this number is very high for LBMs, performance is commonly measured in terms of millions or billions of lattices updated per second, known as MLUPS or GLUPS.

For D3Q19-LBMs, Palabos accesses the 19 populations from a vector f_i^{in} ($i \in \{0, \dots, 18\}$) and writes them back to another vector f_i^{out}, in a two-population implementation fashion. Additionally, two flags are used to (1) identify the location of LB data in the Cartesian mesh and (2) check whether the current cell requires boundary condition treatment. Based on this, $n_t = 2 \cdot 19 \cdot n_{\text{store}} + 8 + 4$ Bytes, with n_{store} being 4 or 8 Bytes depending on whether populations are stored in single or double precision, respectively. This leads to the following peak performance

$$P_{\text{A100}}^{\text{peak,sp}} = B_{\text{A100}}/n_t^{\text{sp}} = 9.481 \text{ GLUPS} \quad \text{and} \quad P_{\text{A100}}^{\text{peak,dp}} = B_{\text{A100}}/n_t^{\text{dp}} = 4.921 \text{ GLUPS} \tag{7}$$

for single precision (sp) and double precision (dp), respectively.

Memory usage, *mem*, is another important factor that impacts the performance of LBMs when running on GPUs (see Fig. 7 in Ref. [11], and Section 7 of Ref. [22]). Generally speaking, the memory usage can be estimated from the ratio between the memory size and the amount of data stored in memory. For D3Q19-LBMs implemented in Palabos, we obtain for a cubic domain

$$mem = \frac{40 \cdot 10^9}{n_t \cdot L^3}, \tag{8}$$

since the memory size of the A100-SXM4 GPU used in this work is 40GB. Assuming $L = 500$, the memory usage is then $mem \approx 51\%$ and 99% for single precision and double precision storage, respectively. In practice, Eq. (8) can be reverted to compute the minimal mesh resolution to perform simulations at optimal performance, as well as, the maximal domain size that can fit the entire memory of the GPU.

Hereafter, the performance model and memory usage are used to better understand performance metrics of the multi-GPU version of Palabos. As performance can slightly vary from one run to another, each measurement is performed three times. The data provided below then consists of the averaged values obtained from these three measurements.

4.2. Single GPU performance: Impact of memory usage and collision model

Let us start with a performance study based on the Taylor-Green vortex benchmark presented in Section 3.1. Fig. 9 illustrates the impact of mesh resolution, and the corresponding memory usage, on performance using BGK and RR collision models. As observed in a previous studies [11,22], performance is an increasing function of the mesh resolution, and the corresponding memory load of the A100 GPU. It exceeds 75% of peak performance when memory usage is greater than 25% (light blue area), and it gradually approaches 80-85% of peak performance as memory usage increases. This trend is consistent for both the simple BGK and the more robust RR collision models, and the peak performance value is close to what one gets with GPU-native approaches [46,91].

In order to check the impact of collision models used in Section 3 on the performance of Palabos, we compare them in Fig. 10. The latter shows that all D3Q19-LBMs achieve between 75% and 85% of peak performance. This indicates that the collision step has minimal impact on performance, provided that (1) the GPU offers sufficient computational power, and (2) collision models are optimized to minimize computations. Both aspects are crucial for gaming-class GPUs, which have significantly reduced computational power for double precision arithmetic. As such, a number of optimization were implemented in Palabos, and they involve unrolling loops, using symmetry rules to reduce the number of terms to compute, and precomputing redundant terms. The interested reader may refer to Section 2.6 in Ref [11] and Section 4.3 in Ref [92] for more details on optimization strategies of the collision step.

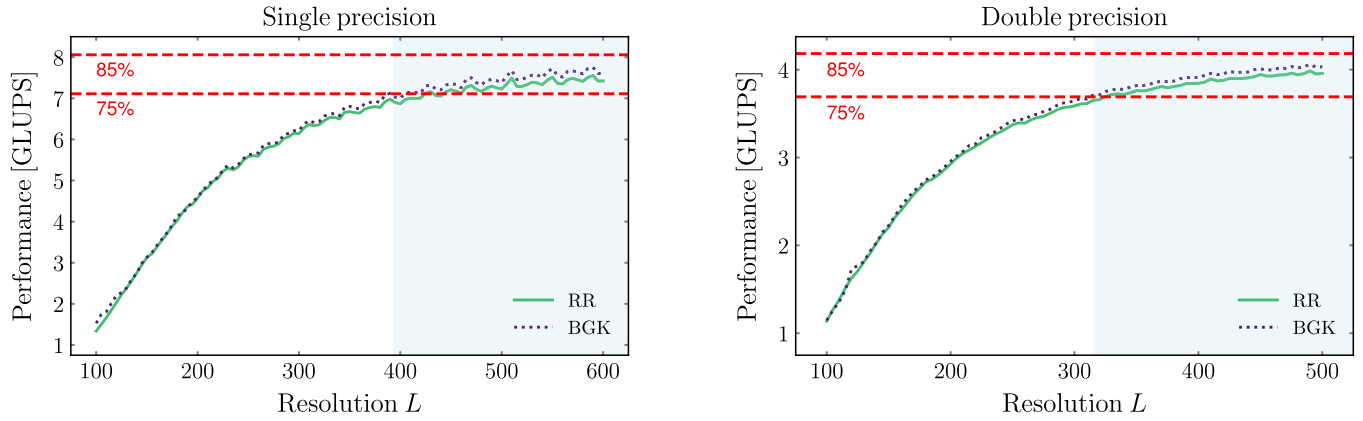


Fig. 9. Mesh resolution impact on the performance obtained on a single A100-SXM4 (40GB) GPU for the Taylor Green vortex simulation. Results are shown for single (left panel) and double (right panel) precision computations, as well as for BGK and RR collision models. The light blue area indicates a memory usage of at least 25% of the total memory capacity of the A100-SXM4 (40GB). The red dashed lines represent 75% and 85% of peak performance. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

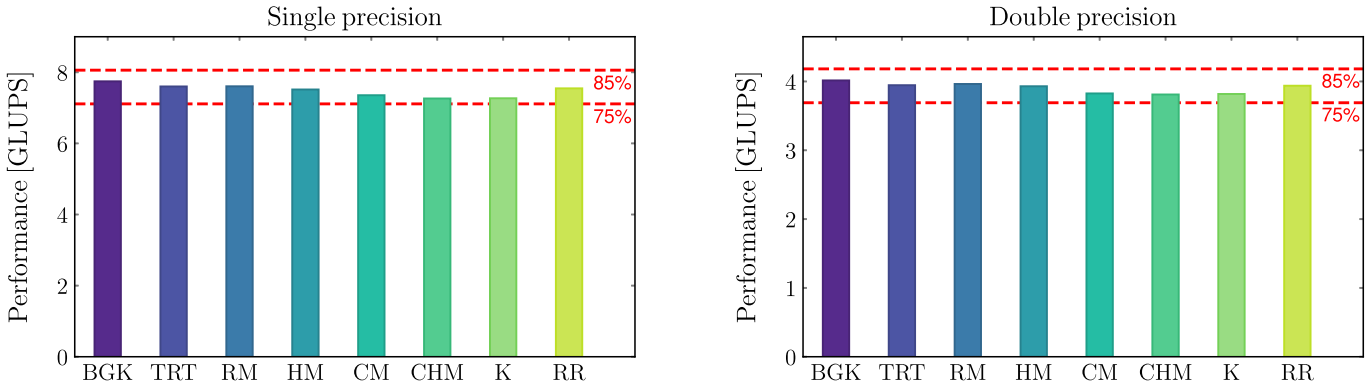


Fig. 10. Collision model impact on the performance obtained on a single A100-SXM4 (40GB) GPU for the Taylor Green vortex simulation. Simulations are performed using $L = 590$ and 490 when single and double precision are used, respectively. The red dashed lines stands for 75 and 85% of the peak performance. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

4.3. Multi-GPU performance: Weak and strong scaling analysis

Below, we aim to quantify the performance of our multi-GPU porting through two specific configurations: weak scaling and strong scaling. The weak scaling study evaluates how effectively the numerical method handles larger problems while maintaining a constant amount of data processed per GPU. Conversely, the strong scaling analysis measures how the simulation time is reduced when the problem size remains fixed, but the number of GPUs increases.

As with the single GPU performance analysis, we use the Taylor-Green vortex case to first evaluate the weak scaling of Palabos on a full DGX-A100-SXM4 (4×40GB). To meet the requirements of a weak scaling study, the simulation domain length L is chosen to (1) end up with optimal performance on a single GPU, and (2) ensure each GPU processes approximately the same number of cells. Choosing $L = 590$ for single precision and $L = 490$ for double precision satisfies the first point, while leaving a safety margin to avoid fully loading the GPU. The second point is achieved by multiplying L by $\sqrt[3]{2}$, $\sqrt[3]{3}$ and $\sqrt[3]{4}$ for two, three and four GPUs, respectively. Corresponding results are gathered in Fig. 11. It is interesting to note that Palabos experiences minimal performance loss when increasing the number of GPUs, even without overlapping communications and computations. Furthermore, the measured weak scaling is linear which means the performance is not deteriorated when the full DGX is used. This result holds true for the BGK, RR and other collision models (not shown here), hence, corroborating the findings presented in Section 4.2. One possible reason for the favorable linear weak scaling of Palabos is that the ratio of communicated to processed data remains relatively constant across all configurations.

In the strong scaling analysis, we start with an optimal domain size and increase the number of GPUs from one to four. Consequently, the effective domain size processed per GPU is $L_{\text{eff}} = \sqrt[3]{L^3/\#\text{GPUs}} < L$. Practically speaking, $L_{\text{eff}} \in \{590, 468, 409, 372\}$ and $L_{\text{eff}} \in \{480, 381, 333, 302\}$ for single and double precision when $\#\text{GPUs} \in \{1, 2, 3, 4\}$. Referring to Fig. 9, one can expect a degradation in performance per GPU due to a reduced memory usage as the number of GPUs increases. This non-linear effect is further expected to add up with the performance loss (10-20%) induced by the non-overlapped data communication, and already observed in the weak scaling study. Fig. 12 presents strong scaling data for the BGK and RR collision models. The results are consistent for both D3Q19-LBMs and confirm the predicted non-linear deterioration in performance. Quantitatively speaking, both loss effects result in a strong scaling efficiency of about 80% for two GPUs and 65% for the full DGX. These remain particularly impressive results for the first multi-GPU release of Palabos, especially considering the complexity and size of the codebase, which exceeds 500,000 lines of code, and the fact that similar trends have been observed when comparing weak and strong scaling properties of GPU-native LBM solvers [46].

4.4. Performance comparison across simulation cases

In this final study, we evaluate the ability of the multi-GPU version of Palabos to handle various configurations in an efficient manner. No-

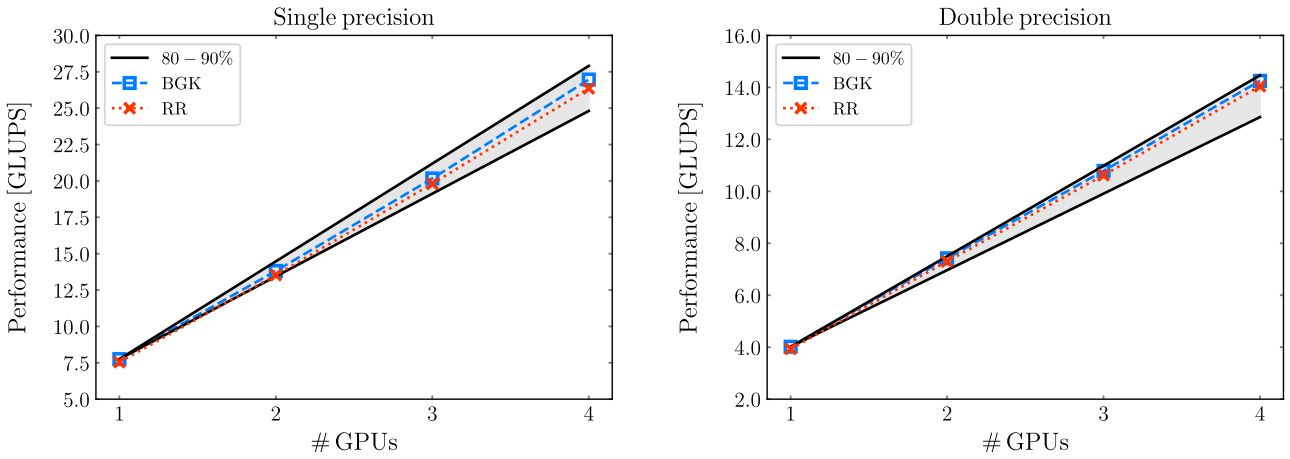


Fig. 11. Weak scaling benchmark performed on the Taylor Green vortex case using a full DGX-A100-SXM4 (4×40GB). The simulation domain length is $L \in \{590, 743, 851, 937\}$ and $L \in \{480, 604, 692, 761\}$ for single and double precision, respectively. Performance remains between 80 and 90% of the ideal weak scaling (grey area).

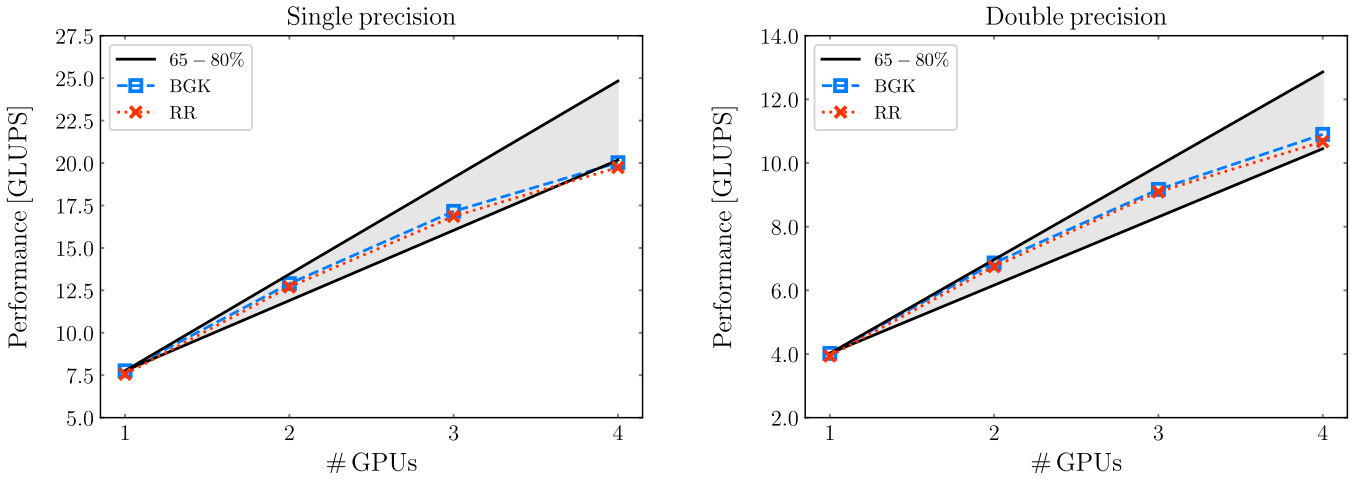


Fig. 12. Strong scaling benchmark performed on the Taylor Green vortex case using a full DGX-A100-SXM4 (4×40GB). The simulation domain length is fixed to $L = 590$ and $L = 480$ for single and double precision, respectively. Performance remains between 65 and 80% of the ideal strong scaling (grey area).

tably, we are interested in quantifying the impact of boundary conditions on performance: periodic for the Taylor-Green vortex (TGV), standard/modified bounce-back for the lid-driven cavity (LDC), and regularized/bounce-back/periodic for the Berea sandstone (BS). Additionally, the BS case will help us evaluate the robustness of the multi-GPU implementation of Palabos in handling heterogeneous treatment of contiguous data in memory, especially inside the sandstone where switching between solid and fluid dynamics is very frequent.

We start with data obtained running the TGV (BGK, $L = 590$), the LDC (BGK, $L = 590$), and the BS (TRT, $L = 400$) cases on a single GPU for single precision. Corresponding results are presented in the top row of Fig. 13. It can be seen that performance is similar for all cases as it remains between 75 and 85% of the peak performance, which is a solid performance, compatible with the one obtained in simpler LBM codes based on C++ parallel algorithms [11]. Moreover, the achieved performance is comparable to that of other open-source, state-of-the-art GPU LBM codes. For uniform-mesh applications on A100-SXM4 (40 GB) GPUs, and using the peak performance (7) as a reference value for this GPU, WaLBerla (CUDA backend) reports a performance of approximately 85% of peak [46], while FluidX3D (OpenCL backend) achieves around 84% [93]. Both values are near the upper bound of the range plotted on Fig. 13, which confirms the efficiency of the Palabos GPU port based on modern C++.

In the weak and strong scaling analyses, discrepancies are observed between the TGV, LDC, and BS cases, as illustrated in the middle and bottom rows of Fig. 13. For all configurations, the weak scaling is almost linear, remaining between 80% and 90% of its ideal value. As previously noted for the TGV case (Section 4.3), the strong scaling is non-linear and ranges from 65 to 80% of its ideal value.

Going into more details, performance discrepancies between configurations are reduced when using double precision, especially in the weak scaling analysis. As expected, periodicity introduces additional communications, leading to more significant performance degradation for the TGV case, compared to LDC and BS, as the number of GPUs increases. Despite the heterogeneous treatments of fluid and solid cells, both LDC and BS perform well, with a slight advantage for LDC. However, the strong scaling performance of BS deteriorates significantly when utilizing three or more GPUs. This decline is primarily due to the lower memory usage resulting from the reduced mesh resolution, which was necessary to stay within the memory limits of the GPU card, as additional memory is required to store the geometry of the sandstone.

In conclusion, these results demonstrate that by utilizing C++ parallel algorithms to port Palabos to GPU, it is possible to achieve near-peak performance on a single A100-SXM4 (40GB) GPU and very good weak and strong scaling on a DGX-A100-SXM4 (4×40GB). This is especially encouraging given that computations and communications have

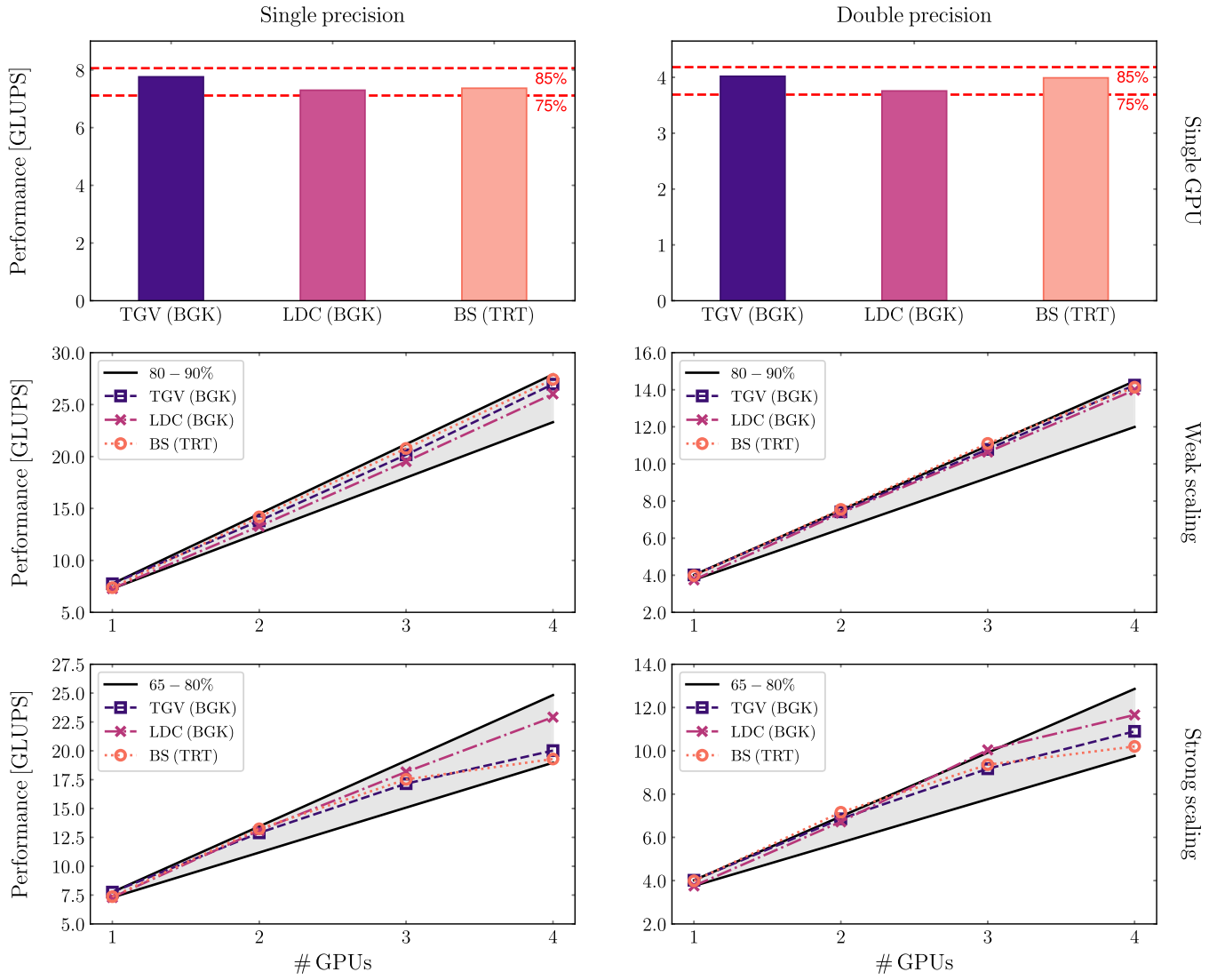


Fig. 13. Performance comparison for all configurations considered in Section 3: Taylor-Green vortex (TGV), lid-driven cavity (LDC), and Berea sandstone (BS). Corresponding simulations are performed using BGK, BGK and TRT collision models. Single GPU, weak scaling, and strong scaling results are gathered in the top, middle and bottom row, respectively. Single GPU performance is measured using $L \in \{590, 590, 400\}$ for single precision, and $\{480, 480, 317\}$ for double precision. For weak and strong scaling, the simulation domain sizes are computed according to the rules described in Section 4.3.

not been overlapped in this first release of the multi-GPU version of Palabos fluid solver.

5. Conclusion

In this work, we have presented the GPU porting of the widely used lattice Boltzmann software library Palabos, using only the standard parallelism available in modern C++. This approach avoids any reliance on external libraries or vendor-specific extensions, which guarantees portability and makes the code easier to maintain across different hardware platforms. The new hybrid software architecture connects the customizable, object-oriented data structures of the CPU version with data-oriented structures optimized for the GPU. This allows existing features to be gradually migrated to the GPU without duplicating large amounts of code.

It is important to emphasize that Palabos has never been limited to being just a simulation tool. From its beginning, it has been designed as a framework for scientists and engineers to develop, test, and improve their own lattice Boltzmann models and algorithms. This is why this foundational philosophy has been meticulously followed throughout the GPU port of its fluid solver. Consequently, the design abstracts the complexity of GPU programming from the end user, offering clear and well-structured mechanisms for implementing new and potentially complex LBM models. Users can thus focus on model development without requiring in-depth knowledge of GPU architectures.

The GPU version of Palabos fluid solver has been validated on several demanding test cases, including the laminar-to-turbulent transition of a Taylor-Green vortex, flow in a lid-driven cavity, and pore-scale flow through a Berea sandstone sample. In all cases, the results show very good to excellent agreement with reference data. Performance tests show that the GPU port achieves near-peak performance on a single GPU and good scaling on multi-GPU systems, even without overlapping communication and computation. This demonstrates that the current porting strategy provides performance comparable to GPU-native codes, while maintaining the flexibility and accessibility that have made Palabos widely used in the scientific community.

The experience gained in this work also serves as a practical example of how complex C++ scientific software can be adapted to GPU execution while keeping the code both understandable and accessible to a wide range of users. Current efforts are focused on porting

additional LBM velocity sets (e.g., D3Q7, D3Q15, D3Q27, D3Q39), as well as `MultiScalarField` objects that are commonly used for the coupling of LBM and finite difference schemes. Future developments include adding overlap of communication and computation to improve multi-GPU performance, extending support to other types of accelerator hardware, and further improving the ease with which users can create new simulations and models. With these advancements, Palabos is ready to remain a leading open-source framework for advanced lattice Boltzmann simulations on current and future high-performance computing systems.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors revised numerous sentences and paragraphs using the online user interfaces of DeepL Write and ChatGPT 4 to enhance grammar, style, and readability. ChatGPT 4 was also used as a productivity tool to produce the source of some of the sketches. After using these services, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Jonas Latt: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Christophe Coreixas:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors would like to express their gratitude to the NVIDIA team for their support and assistance during the initial stages of the GPU porting of the Palabos fluid solver using modern C++. Special thanks are extended to Gonzalo Brito and Jeff Larking for fruitful conversations, technical support, and contributions to the NVIDIA developer two-part article. C.C. acknowledges financial support from BNBU Research Fund nº UICR0700094 - 24 entitled “GPU-accelerated multidisciplinary methods for industrial computational fluid dynamics”.

References

[1] J. Nickolls, I. Buck, M. Garland, K. Skadron, Scalable parallel programming with CUDA: is CUDA the parallel programming model that application developers have been waiting for?, Queue 6 (2008) 40–53.
[2] C heterogeneous-compute interface for portability, 2023. https://rocm.docs.amd.com/projects/HIP/en/latest/.
[3] S. Wienke, P. Springer, C. Terboven, D. Mey, OpenACC - first experiences with real-world applications, in: C. Kaklamanis, T. Papatheodorou, P.G. Spirakis (Eds.), Euro-Par 2012 Parallel Processing, Berlin Heidelberg, Berlin; Heidelberg, Springer, 2012, pp. 859–870.
[4] R. Chandra, R. Menon, L. Dagum, D. Kohr, D. Maydan, J. Mcdonald, Elsevier, 2001.
[5] A. Munshi, The OpenCL specification, in: IEEE Hot Chips 21 Symposium, 2009, pp. 1–314. https://doi.org/10.1109/HOTCHIPS.2009.7478342
[6] A. Alpay, V. Heuveline, SYCL beyond opencl: the architecture, current state and future direction of hipSYCL, in: Proceedings of the International Workshop on OpenCL, IWOCL ’20, number doi:10.1145/3388333.3388658 in the International Workshop on OpenCL, IWOCL ’20New York, NY, USA, Association for Computing Machinery, 2020. https://doi.org/10.1145/3388333.3388658
[7] N. Bell, J. Hoberock, Thrust: a productivity-oriented library for CUDA, in: W. mei, W. Hwu (Eds.), GPU Computing Gems Jade Edition, Applications of, Boston, Morgan Kaufmann, 2012, pp. 359–371. https://doi.org/10.1016/B978-0-12-385963-1.00026-5
[8] H.C. Edwards, C.R. Trott, D. Sunderland, Kokkos: Enabling Manycore Performance Portability Through Polymorphic Memory Access Patterns, 74, Domain-Specific Languages and High-Level Frameworks for High-Performance Computing, 2014.
[9] J. Larkin, No More Porting: Coding for GPUs with Standard C , Fortran, and Python, 2022. NVIDIA GTC (spring), https://www.nvidia.com/en-us/on-demand/session/gtcspring22-s41496/.
[10] J. Larkin, A. Stulova, No More Porting: Accelerated Computing with Standard C , Fortran, and Python, 2024. NVIDIA GTC (spring), https://www.nvidia.com/en-us/on-demand/session/gtc24-s61204/.
[11] J. Latt, C. Coreixas, J. Beny, Cross-platform programming model for many-core lattice Boltzmann simulations, PLoS ONE 16 (2021) 1–29.
[12] C. Coreixas, J. Latt, X. Shan, No more porting: GPU acceleration of lattice Boltzmann and finite difference solvers via C parallel algorithms, in: Symposium on High Fidelity LES/DNS (HiFiLeD) for Industrial Relevant Flow Configurations, 2022.
[13] R.M. Caplan, M.M. Stulajter, J.A. Linker, C. Downs, L.A. Upton, B.K. Jha, R. Attie, C.N. Arge, C.J. Henney, Open-source flux transport (OFT). I. HipFT-High-performance flux transport, Astrophys. J. Suppl. Ser. 278 (2025) 24.
[14] R.M. Caplan, M.M. Stulajter, J.A. Linker, J. Larkin, H.A. Gabb, S. Su, I. Rodriguez, Z. Tschirhart, N. Malaya, Portability of Fortran’s ‘Do Concurrent’ on GPUs, in: SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2024. https://doi.org/10.1109/SCW63240.2024.00240
[15] Y. Asahi, T. Padioleau, G. Latu, J. Bigot, V. Grandgirard, K. Obrejan, Performance portable Vlasov code with C parallel algorithm, in: IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC), 2022, pp. 68–80. https://doi.org/10.1109/P3HPC56579.2022.00012
[16] R. Maggio-Aprile, GPU-based simulation of dense suspensions: coupling the DEM and the, in: LBM in Standard C , Master Thesis, 2023. https://archive-ouverte.unige.ch/unige:174443/.
[17] M. Rambosson, GPU-based implementation of ellipsoids for DEM and LBM using standard C, Technical Report, Master thesis, 2024. https://archive-ouverte.unige.ch/unige:183257/.
[18] R. Maggio-Aprile, M. Rambosson, C. Coreixas, J. Latt, LEDDS: Portable LBM–DEM simulations on GPUs, arXiv:2512.04997 [physics.comp-ph] (2025).
[19] W.-C. Lin, T. Deakin, S. Mcintosh-Smith, Performance modeling, benchmarking and simulation of high performance computer systems (PMBS), 2022, pp. 36–47. Evaluating ISO C parallel algorithms on heterogeneous HPC systems. https://doi.org/10.1109/PMBS56514.2022.00009
[20] M. Haseeb, W. Wei, J. Deslippe, B. Cook, That’s right, the same C, in: STL Asynchronous Parallel Code Runs on CPUs & GPUs, 2023.
[21] W.-C. Lin, S. Mcintosh-Smith, T. Deakin, Preliminary report: initial evaluation of StdPar implementations on AMD GPUs for HPC, Technical Report, 2024, arXiv:2401.02680
[22] C. Coreixas, J. Latt, GPU-based compressible lattice Boltzmann simulations on non-uniform grids using standard C parallelism: from best practices to aerodynamics, aeroacoustics and supersonic flow simulations, Technical Report, 2025, arXiv:2504.04465
[23] J. Latt, O. Malaspinas, D. Kontaxakis, A. Parmigiani, D. Lagrava, F. Brogi, M.B. Belgacem, Y. Thorimbert, S. Leclaire, S. Li, F. Marson, J. Lemus, C. Kotsalos, R. Conradin, C. Coreixas, R. Petkantchin, F. Raynaud, J. Beny, B. Chopard, Palabos: parallel lattice Boltzmann solver, Comput. Math. Appl. 81 (2021) 334–350.
[24] J. Latt, C. Coreixas, G. Brito, J. Larkin, Multi-GPU Programming with Standard Parallel C (Part 1), Technical Report, Technical Report, 2022. https://developer.nvidia.com/blog/multi-gpu-programming-with-standard-parallel-c-part-1/.
[25] J. Latt, C. Coreixas, G. Brito, J. Larkin, Multi-GPU Programming with Standard Parallel C (Part 2), Technical Report, Technical Report, 2022. https://developer.nvidia.com/blog/multi-gpu-programming-with-standard-parallel-c-part-2/.
[26] D. Lagrava, O. Malaspinas, J. Latt, B. Chopard, Advances in multi-domain lattice Boltzmann grid refinement, J. Comput. Phys. 231 (2012) 4808–4822.
[27] F. Brogi, O. Malaspinas, B. Chopard, C. Bonadonna, Hermite regularization of the lattice Boltzmann method for open source computational aeroacoustics, J. Acoust. Soc. Am. 142 (2017) 2332–2345.
[28] C. Lyu, P. Liu, T. Hu, X. Geng, Q. Qu, T. Sun, R.A.D. Akkermans, Hybrid method for wall local refinement in lattice Boltzmann method simulation, Phys. Fluids 35 (2023) 17103.
[29] A. Parmigiani, J. Latt, M.B. Begacem, B. Chopard, A lattice Boltzmann simulation of the Rhône river, Int. J. Mod. Phys. C 24 (2013) 1340008.
[30] H.E. Morrison, A. Leder, Sediment transport in turbulent flows with the lattice Boltzmann method, Comput. Fluids 172 (2018) 340–351.
[31] P. Mercier, M. Grondeau, S. Guillou, J. Thiébot, E. Poizot, Numerical study of the turbulent eddies generated by the seabed roughness. case study at a tidal power site, Appl. Ocean Res. 97 (2020) 102082.
[32] M. Grondeau, S.S. Guillou, J.C. Poirier, P. Mercier, E. Poizot, Y. Méar, Studying the wake of a tidal turbine with an IBM-LBM approach using realistic inflow conditions, Energies 15 (2022)2092.
[33] T. Lolies, N. Gourdain, M. Charlotte, H. Belloc, B. Goldsmith, Numerical Methods For Efficient Fluid-Structure Interaction Simulations of Paragliders, Aerotecnica Missili & Spazio, 2019.
[34] P. Seil, S. Pirker, LBDEMcoupling: open-source power for fluid-particle systems, in: Proceedings of the 7th International Conference on Discrete Element Methods, in the 7th International Conference on Discrete Element Methods, Springer, 2017, pp. 679–686. https://doi.org/10.1007/978-981-10-1926-5_70

[35] A. Parmigiani, S. Faroughi, C. Huber, O. Bachmann, Y. Su, Bubble accumulation and its role in the evolution of magma reservoirs in the upper crust, Nature 532 (2016) 492–495.

[36] J. Lemus, A. Fries, P.A. Jarvis, C. Bonadonna, B. Chopard, J. Latt, Modelling settling-driven gravitational instabilities at the base of volcanic clouds using the lattice Boltzmann method, Front. Earth Sci. 9 (2021) 713175.

[37] S. Leclaire, A. Parmigiani, B. Chopard, J. Latt, Three-dimensional lattice Boltzmann method benchmarks between color-gradient and pseudo-potential immiscible multi-component models, Int. J. Mod. Phys. C 28 (2017) 1750085.

[38] S. Anvari, E. Osei, N. Maftoon, Interactions of platelets with circulating tumor cells contribute to cancer metastasis, Sci. Rep. 11 (2021) 15477.

[39] A. Shodiev, E. Primo, O. Arcelus, M. Chouchane, M. Osenberg, A. Hilger, I. Manke, J. Li, A.A. Franco, Insight on electrolyte infiltration of lithium ion battery electrodes by means of a new three-dimensional-resolved lattice Boltzmann model, Energy Storage Mater. 38 (2021) 80–92.

[40] C. Kotsalos, J. Latt, B. Chopard, Bridging the computational gap between mesoscopic and continuum modeling of red blood cells for fully resolved blood flow, J. Comput. Phys. 398 (2019) 108905.

[41] R. Petkantchin, A. Rousseau, O. Eker, K. Zouaoui Boudjeltia, F. Raynaud, B. Chopard, A simplified mesoscale 3d model for characterizing fibrinolysis under flow conditions, Sci. Rep. 13 (2023) 13681.

[42] R. Conradin, C. Coreixas, J. Latt, B. Chopard, PalaCell2D: a framework for detailed tissue morphogenesis, J. Comput. Sci. 53 (2021) 101353.

[43] J. Latt, C. Coreixas, F. Marson, J.P. D.S. Neto, K. Thyagarajan, B. Chopard, P. From, CPU to GPU in 80 days, in: 30th International Conference on Discrete Simulation of Fluid Dynamics (DSFD), 2021. https://doi.org/10.13140/RG.2.2.10340.71046/1

[44] W. Kim, M. Voss, Multicore desktop programming with intel threading building blocks, IEEE Softw. 28 (2011) 23–31.

[45] J. Tölke, Implementation of a lattice Boltzmann kernel using the compute unified device architecture developed by Nvidia, Comput. Vis. Sci. 13 (2010) 29.

[46] M. Holzer, Code generation in a lattice Boltzmann framework for exascale computing, Technical Report, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU, 2025 Ph.D. thesis, https://open.fau.de/handle/openfau/33506.

[47] O. Malaspinas, Increasing stability and accuracy of the lattice Boltzmann scheme: recursivity and regularization, Technical Report, 2015, arXiv:1505.06900

[48] C. Coreixas, B. Chopard, J. Latt, Comprehensive comparison of collision models in the lattice Boltzmann framework: theoretical investigations, Phys. Rev. E 100 (2019) 33305.

[49] S. Hou, J. Sterling, S. Chen, G.D. Doolen, A lattice Boltzmann subgrid model for high Reynolds number flows, Technical Report, 1994, arXiv:comp-gas/9401004

[50] O. Malaspinas, P. Sagaut, Consistent subgrid scale modelling for lattice Boltzmann methods, J. Fluid Mech. 700 (2012) 514–542.

[51] Nvidia, UCX - communication libraries (NVIDIA multi-node NVLink tuning guide), Online; accessed 2025-01, https://docs.nvidia.com/multi-node-nvlink-systems/multi-node-tuning-guide/ucx.html.

[52] O. M. Project, CUDA - open MPI documentation, online; accessed 2025-01, https://docs.open-mpi.org/en/main/tuning-apps/networking/cuda.html.

[53] J. Choi, Z. Fink, S. White, N. Bhat, D.F. Richards, L.V. Kalé, GPU-aware communication with UCX in parallel programming models: charm , MPI, and Python, in: 2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), IEEE, 2021, pp. 479–488. https://doi.org/10.1109/IPDPSW52791.2021.00079

[54] Y.H. Temuçin, A. Sojoodi, P. Alizadeh, A. Afsahi, Efficient multi-path NVLink/PCIe-aware UCX based collective communication for deep learning, in: 2021 IEEE Symposium on High-Performance Interconnects (HOTI), IEEE, 2021, pp. 1–10. https://doi.org/10.1109/HOTI52880.2021.00018

[55] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, E.M. Viggen, The Lattice Boltzmann Method: Principles and Practice, Springer International Publishing, 2017. https://doi.org/10.1007/978-3-319-44649-3

[56] A.J.C. Ladd, Numerical simulations of particulate suspensions via a discretized Boltzmann equation. part 1. Theoretical foundation, J. Fluid Mech. 271 (1994) 285–309.

[57] A.J.C. Ladd, Numerical simulations of particulate suspensions via a discretized Boltzmann equation. part 2. Numerical results, J. Fluid Mech. 271 (1994) 311–339.

[58] P.A. Skordos, Initial and boundary conditions for the lattice Boltzmann method, Phys. Rev. E 48 (1993) 4823–4842.

[59] M.E. Brachet, D.I. Meiron, S.A. Orszag, B.G. Nickel, R.H. Morf, U. Frisch, Small-scale structure of the Taylor-Green vortex, J. Fluid Mech. 130 (1983) 411–452.

[60] J. Debonis, Solutions of the Taylor-Green Vortex Problem Using High-Resolution Explicit Finite Difference Methods, doi:10.2514/6.2013-382, 2013. https://doi.org/10.2514/6.2013-382

[61] T. Dairay, E. Lamballais, S. Laizet, J.C. Vassilicos, Numerical dissipation vs. subgrid-scale modelling for large eddy simulation, J. Comput. Phys. 337 (2017) 252–274.

[62] S.P. Jammy, C.T. Jacobs, N.D. Sandham, Performance evaluation of explicit finite difference algorithms with varying amounts of computational and memory intensity, J. Comput. Sci. 36 (2019) 100565.

[63] A. Suss, I. Mary, T.L. Garrec, S. Marié, Comprehensive comparison between the lattice Boltzmann and Navier-Stokes methods for aerodynamic and aeroacoustic applications, Comput. Fluids 257 (2023) 105881.

[64] S. Laizet, E. Lamballais, J.C. Vassilicos, T. Dairay, D Taylor-Green vortex Direct Numerical Simulation statistics from Re = 1250 to Re = 20000, 3, 2019. https://doi.org/10.5281/zenodo.2577239

[65] C. Coreixas, G. Wissocq, B. Chopard, J. Latt, Impact of collision models on the physical properties and the stability of lattice Boltzmann methods, Phil. Trans. R. Soc. A 378 (2020) 20190397.

[66] S. Marié, D. Ricot, P. Sagaut, Comparison between lattice Boltzmann method and Navier-Stokes high order schemes for computational aeroacoustics, J. Comput. Phys. 228 (2009) 1056–1070.

[67] G. Wissocq, C. Coreixas, J.-F. Boussuge, Linear stability and isotropy properties of athermal regularized lattice Boltzmann methods, Phys. Rev. E 102 (2020) 53305.

[68] A.K. Prasad, J.R. Koseff, Reynolds number and end-wall effects on a lid-driven cavity flow, Phys. Fluids A Fluid Dyn. 1 (1989) 208–218.

[69] S. Albensoeder, H. Kuhlmann, Accurate three-dimensional lid-driven cavity flow, J. Comput. Phys. 206 (2005) 536–558.

[70] E. Leriche, S. Gavrilakis, Direct numerical simulation of the flow in a lid-driven cubical cavity, Phys. Fluids 12 (2000) 1363–1376.

[71] E. Leriche, Direct numerical simulation in a lid-driven cubical cavity at high Reynolds number by a Chebyshev spectral method, J. Sci. Comput. 27 (2006) 335–345.

[72] L.A. Hegele, A. Scagliarini, M. Sbragaglia, K.K. Mattila, P.C. Philippi, D.F. Puleri, J. Gounley, A. Randles, High-Reynolds-number turbulent cavity flow using the lattice Boltzmann method, Phys. Rev. E 98 (2018) 43302.

[73] M.A. Ferrari, A.T. Franco, J. Hegele, A. L, Turbulent flow inside a cubic lid-driven cavity using moment representation lattice Boltzmann method, Phys. Fluids 36 (2024) 75104.

[74] M.G. Gerritsen, L.J. Durlofsky, Modeling fluid flow in oil reservoirs, Annu. Rev. Fluid Mech. 37 (2005) 211–238.

[75] R. Satti, S.N. Zuklic, D. Bale, N. Koliha, A. Fager, G. Balasubramanian, B. Crouse, D. Freed, Advanced analysis of clean-up and productivity from perforated rocks using computational fluid dynamics, in: SPE Western Regional Meeting, D041S012R003, 2018.

[76] A. Fager, H. Otomo, R. Salazar-Tio, G. Balasubramanian, B Crouse, R. Zhang, H. Chen, J. Schembre-McCabe, Multi-scale digital rock: application of a multi-scale multi-phase workflow to a carbonate reservoir rock, E3S Web Conf. 366, 2023. 01001

[77] B. Xu, A. Yu, Numerical simulation of the gas-solid flow in a fluidized bed by combining discrete particle method with computational fluid dynamics, Chem. Eng. Sci. 52 (1997) 2785–2809.

[78] P. Kieckhefen, S. Pietsch, M. Dosta, S. Heinrich, Possibilities and limits of computational fluid dynamics - discrete element method simulations in process eengineering: a review of recent advancements and future trends, Annu. Rev. Chem. Biomol. Eng. 11 (2020) 397–422.

[79] C. Rettinger, S. Eibl, U. Rüde, B. Vowinckel, Rheology of mobile sediment beds in laminar shear flow: effects of creep and polydispersity, J. Fluid Mech. 932 (2022) 1.

[80] A. Parmigiani, C. Huber, O. Bachmann, B. Chopard, Pore-scale mass and reactant transport in multiphase porous media flows, J. Fluid Mech. 686 (2011) 40–76.

[81] P. Sanematsu, Y. Shen, K. Thompson, T. Yu, Y. Wang, D.-L. Chang, B. Alramahi, A. Takbiri-Borujeni, M. Tyagi, C. Willson, Image-based Stokes flow modeling in bulk proppant packs and propped fractures under high loading stresses, J. Pet. Sci. Eng. 135 (2015) 391–402.

[82] Z. Liu, H. Wu, Pore-scale modeling of immiscible two-phase flow in complex porous media, Appl. Therm. Eng. 93 (2016) 1394–1402.

[83] S. Kelly, H. El-Sobky, C. Torres-Verdín, M.T. Balhoff, Assessing the utility of FIB-SEM images for shale digital rock physics, Adv. Water Resour. 95 (2016) 302–316. Pore scale modeling and experiments.

[84] D. Davudov, R.G. Moghanloo, Y. Zhang, Interplay between pore connectivity and permeability in shale sample, Int. J. Coal Geol. 220 (2020) 103427.

[85] E.R. Ibrahim, M.S. Jouini, F. Bouchaala, J. Gomes, Simulation and validation of porosity and permeability of synthetic and real rock models using three-dimensional printing and digital rock physics, Scientific Reports 6, 2021. 31775–31781

[86] M. Novitska, S. Cassola, T. Schmidt, M. Duhovic, B.I. Basok, D. May, Microscale domain permeability prediction of fiber reinforcement structures based on the lattice Boltzmann method and machine learning, J. Porous Media 28 (2025) 55–69.

[87] H. Dong, M.J. Blunt, Pore-network extraction from micro-computerized-tomography images, Phys. Rev. E 80 (2009) 36307.

[88] Imperial College Consortium on Pore-scale Imaging and Modelling, number doi:10.6084/m9.figshare.1153794.v2 in Berea sandstone, 2014. https://doi.org/10.6084/m9.figshare.1153794.v2

[89] J. Latt, B. Chopard, O. Malaspinas, M. Deville, A. Michler, Straight velocity boundaries in the lattice Boltzmann method, Phys. Rev. E 77 (2008) 56703.

[90] D.H. Kang, E. Yang, T.S. Yun, Stokes-Brinkman flow simulation based on 3-D μ-ct images of porous rock using grayscale pore voxel permeability, Water Resour. Res. 55 (2019) 4448–4464.

[91] M. Holzer, M. Bauer, H. Köstler, U. Rüde, Highly efficient lattice Boltzmann multiphase simulations of immiscible fluids at high-density ratios on CPUs and GPUs through code generation, Int. J. High Perform. Comput. Appl. 35 (2021) 493–509.

[92] F. Hennig, M. Holzer, U. Rüde, Advanced automatic code generation for multiple relaxation-time lattice Boltzmann methods, SIAM J. Sci. Comput. 45 (2023) 233–C254.

[93] M. Lehmann, M.J. Krause, G. Amati, M. Sega, J. Harting, S. Gekle, Accuracy and performance of the lattice Boltzmann method with 64-bit, 32-bit, and customized 16-bit number formats, Phys. Rev. E 106 (2022) 15308.